

NLP Voice Engine API Manual

Version 1.0.0 (rev 1)

修改记录

编号	说明	版本号	修改日期
1	初版	1.0.0 (rev 1)	2020.09.20
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
16			
17			
18			
19			
20			

目 录

一、	简介	7
二、	引擎API使用设计实例	8
三、	知识库管理	9
3.1	提取所有知识库分类	13
3.2	提取知识库分类的路径	13
3.3	增加知识库分类	15
3.4	更改知识库分类	17
3.5	删除一个知识库分类	19
3.6	完全删除一个知识库分类	20
3.7	提取一个知识库分类	20
3.8	提取一个知识库分类（包含分类路径）	21
3.9	提取某分类的下一层子分类	22
3.10	提取某分类的所有子分类	23
3.11	复制知识库分类	24
3.12	按关键字提取知识点	26
3.13	根据分类提取识别的知识点	26
3.14	仅提取当前分类的知识点	27
3.15	按“faq_id”提取知识点	28
3.16	提取所有知识点	29
3.17	提取所有上下文分类中的知识点	29
3.18	提取所有标准型知识点	30
3.19	提取所有知识点的详细数据	30
3.20	新增知识点	31
3.21	删除知识点	32
3.22	更新知识点	33

3.23	新增知识点附件	34
3.24	删除知识点附件	35
3.25	获取知识点附件	35
3.26	批量更新知识库	35
3.27	导出知识库	37
3.28	重置知识库	38
四、	人机交互记录管理	38
4.1	搜索人机交互记录	40
4.2	按搜索条件对人机交互记录进行计数	42
4.3	更新人机交互记录中的“expectedFaqid”	44
4.4	按“expectedFaqid”删除人机交互记录	46
4.5	同步人机交互记录（仅适用于多机版引擎）	46
五、	机器自学习	48
5.1	批量导出训练数据	49
5.2	筛选导出训练数据	50
5.3	批量导入训练数据记录(不含语音文件)	46
5.4	重置训练数据库	54
5.5	触发引擎自学习	54
5.6	触发针对指定分类的引擎自学习	55
5.7	查看引擎自学习状态	56
5.8	同步引擎（仅适用于多机版引擎）	57
5.9	查看同步引擎状态（仅适用于多机版引擎）	57
六、	语音意图识别	58
6.1	上传语音文件进行语音意图识别	58
6.2	下载已识别的语音文件	59
6.3	开启传送语音流	60
6.4	停止传送语音流	62
6.5	取消传送语音流	64

七、	备份管理.....	66
7.1	检查服务状态（仅适用于多机版引擎）	66
7.2	请求下载知识库数据.....	67
7.3	检查下载知识库数据的请求状态.....	67
7.4	下载知识库数据	68
7.5	导入知识库数据	68
7.6	检查导入知识库数据的状态	69
7.7	批量导出详细报表数据.....	70
7.8	请求下载response数据.....	71
7.9	检查下载response数据的请求状态	72
7.10	下载response数据	73
7.11	导入response数据	73
7.12	检查导入response数据的状态	73
7.13	请求下载所有语音文件（仅适用于多机版引擎）	74
7.14	列出可供下载的语音文件包（仅适用于多机版引擎）	75
7.15	下载语音文件包（仅适用于多机版引擎）	76
7.16	删除语音文件包（仅适用于多机版引擎）	76
八、	配置管理.....	77
8.1	更新引擎基本配置参数.....	77
8.2	获取意图识别使用量.....	78
8.3	获取引擎当前时间.....	79
8.4	获取引擎到期时间	79
8.5	获取引擎详细版本	80
8.6	获取引擎操作日志.....	80
8.7	获取引擎数据最后导入导出日志.....	81
九、	高阶引擎功能.....	82
9.1	接口安全认证	82
9.2	分类跳转（Category Switching）	84

附录一： 测试引擎教学	87
附录二： 错误信息说明	89
附录三： 语音活性检测（VAD）参数说明	90
附录四： 语音转文字功能说明.....	93
1. 上传语音文件进行语音转文字	93
2. 语音流单次转换	94
3. 语音流连续转换	97

一、简介

自然语言处理引擎用于自动分析理解非规则化的语音数据。引擎提供以下 API 接口：

- 知识库管理
- 人机交互记录管理
- 训练数据管理及引擎训练
- 语音意图识别
- 静默坐席开发框架
- 备份管理
- 配置管理

以下约定适用于本文档：

- NLP Engine 通过 Cloud 方式提供给 Application Developer（应用开发者）进行应用开发。
- 所有URL前面应加http://<Server_Address>:<Server_Port#>，其中Server_Address和Server_Port#分配给Application Developer独立使用。如被分配https接口，即在URL前应加https://<Server_Address>:<Server_Port#>。
 - ❖ 如：/faq/search/all，如果被分配的Server_Address为nlp.ts.com，Port#为 12345，则该URL的完整应为：http:// nlp.ts.com:12345/faq/search/all。如被分配https接口，则该URL的应为：https:// nlp.ts.com:12345/faq/search/all。
- 如使用 NLP Engine 提供的 Web UI，浏览器建议使用 Google Chrome。
- 请注意本系统所有操作编号为 UTF8，并不接受 BIG5 或 GB 码。还有在编辑下面的 CSV 的例子中，请用 Notepad++ 编辑，然后选择 UTF8 保存。如用 windows 内置的 notepad 选择 UTF8 保存，windows 可能在 csv 中增加一些隐藏符号，从而引起导入问题。

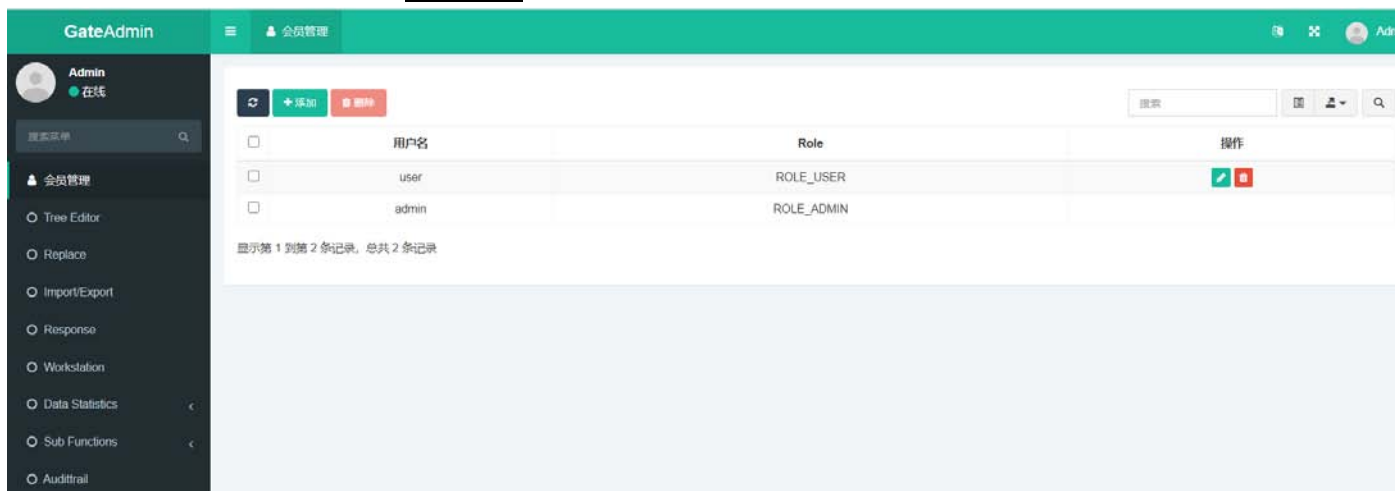
二、引擎 API 使用设计实例

首次登录NLP Engine, 通过访问/login, 即会显示以下接口要求用输入用户账号及密码。



The login interface features a dark blue background. At the top center is a circular placeholder for a user profile picture. Below this are two white input fields: the first is labeled '用户名' (Username) and the second is labeled '密码' (Password). Under the password field is a checkbox labeled '保持会话' (Keep session). At the bottom is a large green button labeled '登录' (Login).

如一切正常, 初次登录后会进入会员管理 用户即可增加, 修改, 删除用户账号, 及更改密码。 如下图所示:



The screenshot shows the 'GateAdmin' user management interface. The top navigation bar is green with 'GateAdmin' and '会员管理' (User Management). The left sidebar contains a search bar and a list of menu items: '会员管理', 'Tree Editor', 'Replace', 'Import/Export', 'Response', 'Workstation', 'Data Statistics', 'Sub Functions', and 'Audittrail'. The main content area displays a table with columns '用户名' (Username), 'Role', and '操作' (Action). The table lists two users: 'user' with role 'ROLE_USER' and 'admin' with role 'ROLE_ADMIN'. Above the table are buttons for '+ 添加' (Add) and '删除' (Delete). Below the table, it says '显示第 1 到第 2 条记录, 总共 2 条记录' (Showing 1 to 2 records, total 2 records).

用户名	Role	操作
user	ROLE_USER	 
admin	ROLE_ADMIN	

请在登录系统后, 尽快更改账号密码。 请注意一点管理员账号不能删除, 而用户名称亦不能重复。

在系统内有两种User Role, 一种是 ROLE_ADMIN, 另一种是ROLE_USER。 主要分别是ROLE_ADMIN能进行用户账号管理及使用/api/faq/clear及/api/response/clear两个清除功能。至于其他API, 如两者有分别, 会在个别API说明。

三、知识库管理

ENGINE 的知识库为树状结构，由分类和知识点组成。分类将知识点分门别类，使得知识库易于管理。建立知识库时，分类有多重类型可以选择，以用于处理不同的知识结构。引擎主要有以下六种分类模式：

分类类型	主要用途
标准 (Standard)	用于一般自然语言文本处理, 主要和其他分类不同之处是当引擎在 ROOT 分类被查询时(单元六接口), 引擎会搜寻「标准分类」及「细化型分类」中的知识点。如果在那个分类(question_id)下搜寻, 则只会搜寻该分类及其所有子分类中的知识点。但如果答案在「标准」分类中, 系统是不会有(followUp)跟进回复。详细关于(followUp)跟进回复, 会在本文件「高阶引擎功能」单元再详细解释。 基本上除非有特定用途, 分类类型一般只需要选择「标准」即可。一个「标准型分类」下, 无论开了多少个「标准型子分类」, 引擎被查询时会将所有「标准型分类」的知识点(意图)统一处理, 母或子分类都不会有什么区别。
枚举(Enum)	主要是用于「顺序型问卷分类」, 一般引擎在被查询时一般不会访问「枚举分类」下的知识点(意图), 除非被主动指示在该类型下寻找。详细「枚举分类」如何应用在「顺序型问卷」会在本文件「高阶引擎功能」单元中说明。
树状型问卷 (Questionnaire_Branching)	这是引擎中的高阶功能, 适用于做「树状型」形式的问卷调查的「上下文功能」, 简单例子如下: ----- 【范例: 甲公司服务调查问卷】 问题一: 你满意我司服务吗? 1. 满意(跳往问题二) 2. 不满意(跳往问题三) 问题二: 你满意什么服务呢? 1. 售前服务(跳往问题四) 2. 售后服务(跳往问题五) 3. 其他 (跳往问题六) 问题三: 你不满意原因是什么呢? 1. 职员态度(跳往问题七) 2. 产品价格(跳往问题八)

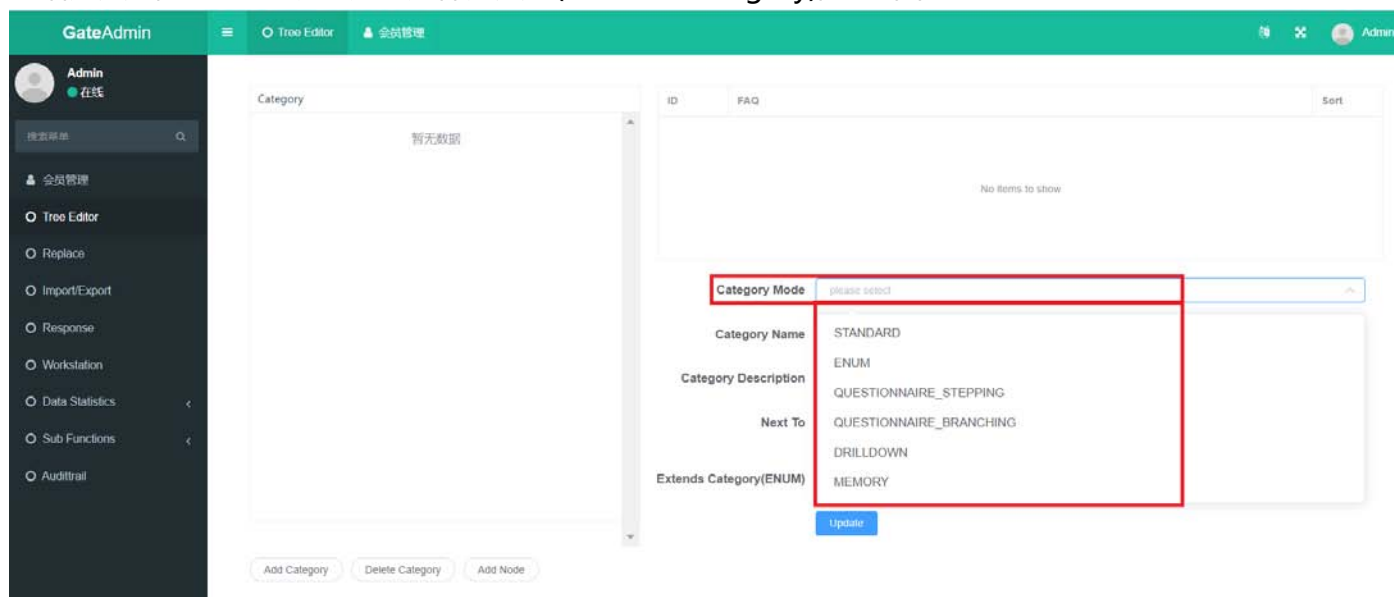
	3. 其他(跳往问题九) ----- 引擎被查询时访问到「树状型问卷分类」时, 系统会在回复时加上“followUp”在回复内, 用意是指示查询者答案会可能在那个「树状型问卷子分类」中, 让查询者利用这个子分类继续搜索, 直至得出最终答案。 还有一点和「标准型分类」不同之处, 是「母分类」和子分类中的知识点(意图)是有层次的概念, 「标准型分类」是没有的。当查询「树状型问卷分类」时, 引擎只会寻找这个「树状型问卷分类」下一层的知识点, 其他同层分类或上层分类的知识点, 是不会被包括在搜寻的范围内, 「标准型分类」中的知识点被搜寻时, 不理上层下层, 都会一起包括在搜寻范围内。 详细关于「树状型问卷」会在本文件「高阶引擎功能」单元再详细解释。
顺序型问卷 (Questionnaire_Stepping)	这是引擎中的高阶功能, 适用于做「顺序型」形式的问卷调查的「上下文功能」, 简单例子如下: ----- 【范例: 购买机票系统】 问题一: 起飞地点是什么? 问题二: 降落地点是什么? 问题三: 起飞日期? 问题四: 客舱等级? ----- 和「树状型问卷」不同之处, 无论使用者如何回答, 问题都会顺序问下去, 不会因为回答跳到其他分类的知识点之中, 引擎回复的 followUp 也一定只会是下一层的分类, 详情如何操作会在本文中的「高阶引擎功能」说明。 还有一点要注意是「顺序型问卷分类」可和「枚举型分类」。就如上面的范例, 起飞地点和降落地点, 其实回复都是城市, 开发者可创造一个关于城市的「枚举型分类」, 然后链接上的「顺序型问卷分类」中, 以减少开发者重复制

	造相似知识点的工作量。如何链接「顺序型问卷分类」的方法会在「高阶引擎功能」说明。
细化(Drilldown)	「细化型分类」的用法及操作大致和「树状型问卷分类」一样, 惟一不同之处是当查询「细化型分类」的知识点时, 引擎会寻找该「细化型分类」之下所有分类的知识点, 无论下面子分类再下面还有多少子分类, 都会放在搜寻范围内。而搜寻「树状型问卷分类」只是会搜寻下一层的子分类, 子分类再下一层的子分类中的知识点, 不会在搜寻范围内。
记忆(Memory)	「记忆型分类」是「顺序型问卷分类」的功能延伸, 只能结合「顺序型问卷分类」一起使用。「记忆型分类」用作记忆整个「顺序型问卷分类」中各层的答案, 然后据此再给出特定答案。 使用方法是在「顺序型问卷」最后的一层分类, 建立一个「记忆型分类」及增加所需的知识点。每当用户查询包含「记忆型分类」的「顺序型问卷分类」时, 系统会逐层记录答案对应的 faqid 和 questionId。当查询到「记忆型分类」时, 系统会自动将之前各层答案对应的 faqid 和 questionId 组合成一个「合成问」, 然后用「合成问」代替用户输入的自然问题访问引擎。用户需要在 Report 功能中选择对应的 Expected Faqid 训练引擎。引擎经过训练后会懂得关联用户在「顺序型问卷」所选的多项答案和「记忆型分类」中下的意图。链接「顺序型问卷分类」的详细方法会在「高阶引擎功能」再说明。

不同分类的具体操作方法略有不同，以下列明六种分类操作的不同之处：

	标准型	枚举型	顺序型问卷	树状型问卷	细化型	记忆型
是否需要在建立分类时同时建立依附在分类上的特殊知识点。 (如下图蓝框中的特殊知识点, 须和分类一起创建。)	不需要	不需要	需要	需要	需要	不需要
是否可以在分类内建立一般知识点。 (如下图红框内的知识点, 不需要在创建分类时一起创立。)	可以	可以	可以	不可以	可以	可以
是否可以在分类内建立子分类。	可以	可以	可以	可以	可以	不可以
在分类内是否可以有多于一个同层的子分类。 (如下图「树状型分类」中, 有两个同层次的「子分类一」及「子分类二」。)	可以	可以	不可以	可以	可以	不可以
当开发者用 QUERY API 指明搜寻该分类的知识点时, 引擎的搜寻范围。	会搜索该分类下, 所有子分类不论多少层中的所有识点。	会搜索该分类的知识点, 以及 extends category 下的知识点。	只会搜索当前分类中的知识点, 该分类下一层的子分类中的知识点不会被搜索。	只会搜索该分类下一层所有子分类中的特殊知识点(依附在分类的知识点)。	会搜索该分类下, 所有子分类不论多少层中的所有识点。	只会搜索该「分类中」的知识点。

新增分类时, 还有一个功能为「延伸分类」(Extends Category), 如下图:



一般情况, 这个分类延伸是不用填上。但如果开发人员想将某个分类和另一个「枚举型分类」连接一起, 只要在这个下拉菜单选上该「枚举型分类」, 两个分类立即会结合在一起, 效果会是当引擎搜寻这个连接了「枚举型分类」的分类时, 会和被连接的「枚举型分类」的知识点也一并搜寻, 并给予相关答案。这个功能的应用场景, 通常会有一些知识点会在各分类都要用到, 但又不想在各分类也同时加上这些知识点。在这个情况下, 只要将这一类会被共享的知识点都放在同一个「枚举型分类」下, 然后在各分类上都选上这个(Extends Category), 加入共享的「枚举型分类」, 那在使用单元六中的 API 搜寻答案时, 会将该分类及被选上的「枚举型分类」都一并搜索。

建立分类后, 就是利用 API 处理各种知识库管理。以下为详细的 API 接口介绍:

3.1 提取所有知识库分类

URL	/api/category/all
HTTP Method	Get
功能	从树状知识库中提取所有分类纪录。
输出	数据类型: JSON 数组 【范例】 <pre>[{ "id": 0, "name": "ROOT", "parentid": 0,</pre>

	<pre> "description": null, "switching": null, "extendsEnumCategory": null, "faq": null, "mode": "STANDARD", "validEnumAttach": true }] </pre>
--	--

3.2 提取知识库分类的路径

URL	/api/category/<category_id>
HTTP Method	Get
功能	系统会利用<category_id>在知识库寻找分类的路径, 及该分类下的所有子分类的路径, 输出结果是按"categorypath"进行升序排序。 <category_id>支持以下二个格式: • 分类路径 (Category Path) , 例如"ROOT->S1", 用"->"来分隔分类名称。 • 分类 ID (Category ID) ;
输出	数据类型: JSON 数组 【范例】 <pre> [{ "categoryid": 1, "name": "ROOT", "parentid": 0, "categorypath": "ROOT" }, { "categoryid": 2, "name": "S1", "parentid": 1, "categorypath": "ROOT->S1" }] </pre>

3.3 增加知识库分类

URL	/api/category/add
HTTP Method	Post
功能	增加知识库分类到从树状知识库中。
输入	数据类型: JSON 对象 - name: 任何 UTF8 文字, 是知识库中的分类名称。(同一层知识库分类名称不能相同) - parentid: 任何系统存在的分类 ID 数字, 是向系统说明新增分类放在那一个存在分类之下。(必须存在) - description: 任何 UTF8 文字, 详细形容分类说明。 - mode: 是该分类的种类, 现在有五种分类种类: • STANDARD (标准型) • ENUM (枚举型) • QUESTIONNAIRE_STEPPING(顺序型问卷) • QUESTIONNAIRE_BRANCHING(树状型问卷) • DRILLDOWN(细化型) • MEMORY(记忆型) - faq: 是 array of hash (哈希数组)。「标准型」及「枚举型」的分类,只上传 null 值即可。至于「顺序型问卷」、「树状型问卷」、「细化型」类的分类,则需要上传该分类的知识点。下面会有范例示范。当中 id 是指 FAQID, 在整个系统要惟一值。 - extendsEnumCategory: 是 array of hash (哈希数组)。是一个分类的 array of hash。当中和一般分类一样, 有 id, name, parentid, description, mode, faq, extendsEnumCategory, 当中惟一要注意是只有"mode":"ENUM", 才可以附加上在分类上。 【增加「标准型分类」范例】 <pre>{ "name": "Category Name", "parentid": "2", "description": "Category Desc", "faq": null "mode": "STANDARD" "extendsEnumCategory":null }</pre> 【增加「顺序型分类」范例】

	<pre> { "id": 6, "name": "S1", "parentid": 1, "description": null, "switching": null, "extendsEnumCategory": { "id": 5, "name": "E", "parentid": 1, "description": null, "switching": null, "extendsEnumCategory": null, "faq": null, "mode": "ENUM" }, "faq": { "id": "S1", "categoryid": 1, "question": "S1", "answer": "S1", "attachments": [] }, "mode": "QUESTIONNAIRE_STEPPING" } </pre>
输出	数据格式: JSON 对象 - MESSAGE: 是新增分类的 Category ID。 【成功范例】 <pre> { "MESSAGE": 16, "STATUS": true } </pre> 【失败范例】 <pre> { "STATUS": false, "MESSAGE": "DUPLICATE_CATEGORY_NAME:S1" } </pre> 或 <pre> { "MESSAGE": "PARENT_NOT_FOUND", "STATUS": false } </pre>

}

3.4 更改知识库分类

URL	/api/category/update
HTTP Method	Post
功能	更改知识库分类。
输入	数据类型: JSON 数组 - id: 需要被改动的分类的 category id, 此 id 必须在知识库存在。此 id 是不能被 update, 除此 id 外的所有内容则可被更新 - name: 任何 UTF8 文字, 是知识库中的分类名称。 - parentid: 任何系统存在的分类 ID 数字, 是向系统说明新增分类放在那一个存在分类之下。 - description: 任何 UTF8 文字, 详细形容分类说明。 - mode: 是该分类的种类, 现在有五种分类种类: • STANDARD (标准型) • ENUM (枚举型) • QUESTIONNAIRE_STEPPING(顺序型问卷) • QUESTIONNAIRE_BRANCHING(树状型问卷) • DRILLDOWN(细化型) • MEMORY(记忆型) - faq: 是 array of hash (哈希数组)。「标准型」及「枚举型」的分类,只上传 null 值即可。至于「顺序型问卷」、「树状型问卷」、「细化型」类的分类,则需要上传该分类的知识点。下面会有范例示范。当中 id 是指 FAQID, 在...整个系统要惟一值。 - extendsEnumCategory: 是 array of hash (哈希数组)。是一个分类的 array of hash。当中和一般分类一样, 有 id, name, parentid, description, mode, faq, extendsEnumCategory, 当中惟一要注意是只有"mode":"ENUM", 才可以附加上在分类上。 要注意除了 id 外, 如果其他字段没有值, 则会视作 null 更新到分类内, 故此用此 API 操作时要极度小心, 最好是先用/api/category/get/拿到将被更改分类的原来数据, 然后用上面的数据更改分类资料, 以保证需要的数据不会被意外地改为 null value。 【增加「标准型分类」范例】 <pre>{</pre>

	<pre> "id": "4", "name": "Category Name", "parentid": "2", "description": "Category Desc", "faq": null "mode": "STANDARD" "extendsEnumCategory": null } </pre> 【增加「顺序型分类」范例】 <pre> { "id": 6, "name": "S1", "parentid": 1, "description": null, "switching": null, "extendsEnumCategory": { "id": 5, "name": "E", "parentid": 1, "description": null, "switching": null, "extendsEnumCategory": null, "faq": null, "mode": "ENUM" }, "faq": { "id": "S1", "categoryid": 1, "question": "S1", "answer": "S1", "attachments": [] }, "mode": "QUESTIONNAIRE_STEPPING", "followUp": "ROOT->S1" } </pre>
输出	数据格式: JSON 对象 - MESSAGE: 是新增分类的 Category ID。 【成功范例】 <pre> { "STATUS": true, "MESSAGE": 16 } </pre>

【失败范例】

```
{
  "MESSAGE": "ID_NOT_FOUND",
  "STATUS": false
}
```

3.5 删除一个知识库分类

URL	/api/category/del/<category_id>
HTTP Method	Get
功能	删除知识库分类。系统会利用<category_id>在知识库寻找要被删除的分类。开发者需要确保该分类下没有其他子分类及知识点，否则引擎会返回 400 错误信息。 <category_id>支持以下三个格式： - 分类路径（Category Path），例如“ROOT->Category One”，用“->”来分隔分类名称。 - 分类 ID（Category ID）； - 分类名称（Category Name）：如果分类名称在系统中不唯一，则分类名称不适用。
输出	数据格式：JSON 对象 系统会输出被删除的分类中所有数据。如用 API 删除分类要小心被删除下的分类下面有没有剩余的「知识点」及子分类，开发者需要自行用 API 将分类下的数据删除，最后才删除该分类。如被删除分类还有子分类及知识点，系统会回报 error 400。 【范例】 <pre>{ "id": 16, "name": "Test Category 1", "parentid": 2, "description": "Test Category1", "faq": { "id": "Test Category 1", "categoryid": 2, "question": "Test Category Question", "answer": "Test Category Question", "attachments": [] }, "mode": "DRILLDOWN" }</pre>

3.6 完全删除一个知识库分类

URL	/api/category/del/<category_id>/recursive
HTTP Method	Get
功能	删除知识库分类。系统会利用<category_id>在知识库寻找要被删除的分类, 及该分类下的所有子分类及知识点。 <category_id>支持以下三个格式: • 分类路径 (Category Path) , 例如"ROOT->Category One", 用"->"来分隔分类名称。 • 分类 ID (Category ID) ; • 分类名称 (Category Name) : 如果分类名称在系统中不唯一, 则分类名称不适用。
输出	数据格式: JSON 对象 系统会输出被删除的分类中所有数据。如该分类下还有其他子分类被删除, 则不会列出。 【范例】 <pre>{ id: 9, name: "test1", parentid: 8, description: "test", extendsEnumCategory: null, faq: null, mode: "STANDARD" }</pre>

3.7 提取一个知识库分类

URL	/api/category/get/<category_id>
HTTP Method	Get
功能	提取知识库分类资料。系统会利用<category_id>在知识库搜寻出该分类资料。 <category_id>支持以下三个格式: • 分类路径 (Category Path) , 例如"ROOT->Category One", 用"->"来分隔分类名称。 • 分类 ID (Category ID) ; • 分类名称 (Category Name) : 如果分类名称在系统中不唯一, 则分类名称不适用。
输出	数据格式: JSON 对象

【范例】

```
{
  "id": 16,
  "name": "Test Category 1",
  "parentid": 2,
  "description": "Test Category1",
  "faq": {
    "id": "Test Category 1",
    "categoryid": 2,
    "question": "Test Category Question",
    "answer": "Test Category Question",
    "attachments": [ ]
  },
  "mode": "DRILLDOWN"
}
```

3.8 提取一个知识库分类（包含分类路径）

URL	/api/category/get/<category_id>/extended
HTTP Method	Get
功能	提取知识库分类资料。系统会利用<category_id>在知识库搜寻出该分类资料。其中包含该分类的分类路径。 <category_id>支持以下三个格式: - 分类路径 (Category Path) , 例如"ROOT->Category One", 用"->"来分隔分类名称。 - 分类 ID (Category ID) ; - 分类名称 (Category Name) : 如果分类名称在系统中不唯一, 则分类名称不适用。
输出	数据格式: JSON 对象 如果<category_id>不存在, 引擎不会有任何输出, http 状态码为 404。 【范例】 <pre>{ "id": 135, "name": "STEPPING_1", "parentid": 133, "description": null, "switching": null, "extendsEnumCategory": { "id": 137,</pre>

```

    "name": "ENUM",
    "parentid": 133,
    "description": null,
    "switching": null,
    "extendsEnumCategory": null,
    "faq": null,
    "mode": "ENUM"
  },
  "faq": {
    "id": "STEPPING_1_FAQ",
    "categoryid": 133,
    "question": "STEPPING_1_Q",
    "answer": "STEPPING_1_A",
    "attachments": []
  },
  "mode": "QUESTIONNAIRE_STEPPING",
  "followUp": "ROOT->STEPPING_1"
}

```

3.9 提取某分类的下一层子分类

URL	/api/category/list/<category_id>/sub
HTTP Method	Get
功能	提取知识库中某分类的下一层子分类的资料。系统会利用<category_id>在知识库搜寻出该分类的下一层子分类的资料。 <category_id>支持以下三个格式: • 分类路径 (Category Path) , 例如"ROOT->Category One", 用"->"来分隔分类名称。 • 分类 ID (Category ID) ; • 分类名称 (Category Name) : 如果分类名称在系统中不唯一, 则分类名称不适用。
输出	数据格式: JSON 数组 如果<category_id>不存在, 引擎不会有任何输出, http 状态码为 404。 【范例】 <pre> [{ "id": 134, "name": "STANDARD", "parentid": 133, "description": null, "switching": null, "extendsEnumCategory": null, </pre>

```

    "faq": null,
    "mode": "STANDARD"
  },
  {
    "id": 135,
    "name": "STEPPING_1",
    "parentid": 133,
    "description": null,
    "switching": null,
    "extendsEnumCategory": {
      "id": 137,
      "name": "ENUM",
      "parentid": 133,
      "description": null,
      "switching": null,
      "extendsEnumCategory": null,
      "faq": null,
      "mode": "ENUM"
    },
    "faq": {
      "id": "STEPPING_1_FAQ",
      "categoryid": 133,
      "question": "STEPPING_1_Q",
      "answer": "STEPPING_1_A",
      "attachments": []
    },
    "mode": "QUESTIONNAIRE_STEPPING"
  }
]

```

3.10 提取某分类的所有子分类

URL	/api/category/list/<category_id>/allsub
HTTP Method	Get
功能	提取知识库中某分类的所有子分类的资料。系统会利用<category_id>在知识库搜寻出该分类的所有子分类的资料。 <category_id>支持以下三个格式: - 分类路径 (Category Path) , 例如"ROOT->Category One", 用"->"来分隔分类名称。 - 分类 ID (Category ID) ; - 分类名称 (Category Name) : 如果分类名称在系统中不唯一, 则分类名称不适用。
输出	数据格式: JSON 数组

如果<category_id>不存在, 引擎不会返回任何信息, http 状态码为 404。

【范例】

```
[
  {
    "id": 134,
    "name": "STANDARD",
    "parentid": 133,
    "description": null,
    "switching": null,
    "extendsEnumCategory": null,
    "faq": null,
    "mode": "STANDARD"
  },
  {
    "id": 135,
    "name": "STEPPING_1",
    "parentid": 133,
    "description": null,
    "switching": null,
    "extendsEnumCategory": {
      "id": 137,
      "name": "ENUM",
      "parentid": 133,
      "description": null,
      "switching": null,
      "extendsEnumCategory": null,
      "faq": null,
      "mode": "ENUM"
    },
    "faq": {
      "id": "STEPPING_1_FAQ",
      "categoryid": 133,
      "question": "STEPPING_1_Q",
      "answer": "STEPPING_1_A",
      "attachments": []
    },
    "mode": "QUESTIONNAIRE_STEPPING"
  }
]
```

3.11 复制知识库分类

URL	/api/category/duplicate
HTTP Method	Post

功能	将一个现有的分类及其下的所有知识点复制到一个现有的指定分类路径下。
输入	数据格式: JSON 对象 - oldQid: 需要被复制的分类的分类路径, 例如“ROOT->A->B”。 - newQid: 目标分类的分类路径, 例如“ROOT->A->B”。该目标分类的「父分类」必须为系统中现有的分类。 - faqidPrefix (optional): 任何 UTF8 文字, 为目标分类中所有 Faqid 添加前缀。 - faqidSuffix (optional): 任何 UTF8 文字, 为目标分类中所有 Faqid 添加后缀。 注意: 至少需要使用 faqidPrefix 和 faqidSuffix 两个字段中的一个。 【范例】 <pre>{ "oldQid": "ROOT->TEST", "newQid": "ROOT->DUP1->DUP2->TEST", "faqidPrefix": "DUP1_", "faqidSuffix": "new" }</pre>
输出	数据格式: JSON 对象 - MESSAGE: 返回该指令的执行消息。 - RESULT: 返回该指令的结果, 如果该指令执行失败, 该字段不会显示。 - STATUS: 返回该指令的执行状态, true 或 false。 【范例】 <pre>{ "MESSAGE": "DONE", "RESULT": [{ "id": 5, "name": "TEST", "parentid": 4, "description": null, "switching": null, "extendsEnumCategory": null, "faq": null, "mode": "STANDARD" }], "STATUS": true }</pre>

3.12 按关键字提取知识点

URL	/api/faq/search/text/<search_text>
HTTP Method	Get
功能	从树状知识库中提取知识记录。用<search_text>去寻找知识点中的标准问及标准答, 并用 JSON 格式回复用户寻找结果。 <search_text>: 任何 UTF-8 字符。
输出	数据格式: JSON 数组 【范例】 <pre>[{ "id": 1101, "categoryid": 3, "question": "Question 1", "answer": "Answer 1", "haveChildren": false, "sort": 999, "faqTag": [], "attachments": [] }]</pre>

3.13 根据分类提取识别的知识点

URL	/api/faq/search/<category_id>
HTTP Method	Get
功能	用<category_id>去寻找知识点(意图)中的标准问及标准答, 并用 JSON 格式回复用户寻找结果。请注意上下文分类中的特殊知识点属于父分类也会提取出来。 不同分类的提取说明: 标准型:会搜索该分类下, 所有子分类不论多少层中的所有识点。 细化型:会搜索该分类下, 所有子分类不论多少层中的所有识点。 顺序型:只会搜索当前分类中的知识点,该分类下一层的子分类中的知识点不会被搜索。 树状型:只会搜索该分类下一层所有子分类中的特殊知识点(依附在分类的知识点)。 枚举型:会搜索该分类的 faq, 以及 extends category 下的 faq。 记忆型:只会搜索该「分类中」的知识点。

	<category_id>支持以下三个格式: • 分类路径 (Category Path) , 例如"ROOT->Category One", 用"->"来分隔分类名称。 • 分类 ID (Category ID) ; • 分类名称 (Category Name) : 如果分类名称在系统中不唯一, 则分类名称不适用。
输出	数据格式: JSON 数组 【范例】 <pre>[{ "id": "D1", "categoryid": 32, "question": "D1", "answer": "D1", "attachments": [] }]</pre>

3.14 仅提取当前分类的知识点

URL	/api/faq/get/<category_id>
HTTP Method	Get
功能	仅提取当前分类的知识点, 并用 JSON 格式回复用户寻找结果。请注意上下文分类中的特殊知识点不会提出来。 <category_id>支持以下三个格式: • 分类路径 (Category Path) , 例如"ROOT->Category One", 用"->"来分隔分类名称。 • 分类 ID (Category ID) ; • 分类名称 (Category Name) : 如果分类名称在系统中不唯一, 则分类名称不适用。
输出	数据格式: JSON 数组 【范例】 <pre>[{ "id": "D1", "categoryid": 32, "question": "D1", "answer": "D1", "attachments": [] }]</pre>

3.15 按“faq_id”提取知识点

URL	/api/faq/id/<faq_id>
HTTP Method	Get
功能	按<faq_id>从知识库中提取知识点。对于上下文分类中的特殊知识点，系统将会返回对应的分类信息，其中包含该知识点信息。 <faq_id>: 任何 UTF-8 字符。
输出	数据格式: JSON 对象 【提取「标准型分类」中知识点的范例】 <pre>{ "id": "1", "categoryid": 33, "question": "1", "answer": "1", "attachments": [{ "id": 11, "filename": "test.wav" }] }</pre> 【提取「顺序型分类」中知识点的范例】 <pre>{ "id": "S2", "categoryid": 38, "question": "S2", "answer": "S2", "attachments": [], "linkedCategory": { "id": 39, "name": "S2", "parentid": 38, "description": null, "switching": null, "extendsEnumCategory": null, "faq": { "id": "S2", "categoryid": 38, "question": "S2", "answer": "S2", "attachments": [{ "id": 11,</pre>

	<pre> "filename": "123.wav" } }, "mode": "QUESTIONNAIRE_STEPPING" }, "linkedFollowUp": "ROOT->S1->S2" } </pre>
--	--

3.16 提取所有知识点

URL	/api/faq/all
HTTP Method	Get
功能	从树状知识库中提取所有知识点的资料。
输出	数据类型：JSON 数组 【范例】 <pre> [{ "id": "1", "categoryid": 33, "question": "1", "answer": "1", "attachments": [] }] </pre>

3.17 提取所有上下文分类中的知识点

URL	/api/faq/category
HTTP Method	Get
功能	从树状知识库中提取依附在上下文分类中的知识点的资料。
输出	数据类型：JSON 数组 【范例】 <pre> [{ "id": "DRILLDOWN_FAQ", "categoryid": 32, "question": "DRILLDOWN_FAQ", "answer": "DRILLDOWN_FAQ", "attachments": [] }] </pre>

	}]
--	--------

3.18 提取所有标准型知识点

URL	/api/faq/standard
HTTP Method	Get
功能	从知识库中提取标准型知识点的资料, 不包括依附在上下文分类中的知识点。
输出	数据类型: JSON 数组 【范例】 <pre>[{ "id": "1", "categoryid": 33, "question": "1", "answer": "1", "attachments": [] }]</pre>

3.19 提取所有知识点的详细数据

URL	/api/faq/all/extended
HTTP Method	Get
功能	从树状知识库中提取所有知识点的详细资料。
输出	数据类型: JSON 数组 【范例】 <pre>[{ "id": "1", "categoryName": "A", "parentCategoryid": 32, "categoryid": 33, "answer": "1", "attachments": [], "question": "1", "parentCategoryName": "ROOT" }]</pre>

	}]
--	--------

3.20 新增知识点

URL	/api/faq/add
HTTP Method	Post
功能	按规定的 JSON 格式, 对知识库进行处理更新。
输入	数据类型: JSON 数组 - id: 任何 UTF8 文字, 是开发者提供给引擎的惟一值, 只可输入数字或字母。 - question: 任何 UTF8 文字, 是知识点(意图)的标准问。 - answer: 任何 UTF8 文字, 是知识点(意图)的标准答。 - category(optional): 任何 UTF-8 文字, 是知识库中的分类。支持以下三种格式: • 分类路径 (Category Path) , 各层分类名称之间用“->”来分隔; • 分类 ID (Category ID) ; • 分类名称 (Category Name) : 如果分类名称在系统中不唯一, 则分类名称不适用。 如果不指明 category, 系统会默认在 ROOT 分类下添加该知识点。 【范例】 <pre>[{ "id": "1", "question": "xxxxxx", "answer": "你好吗?", "responseId": "1234", "category": "ROOT" }]</pre>
输出	数据类型: JSON 数组 【成功范例】 <pre>[{ "faqId": "99", "responseId": "1234" }]</pre> 或

```
[
  {
    "faqId": "99"
  }
]

【失败范例】

[
  {
    "STATUS": "DUPLICATED_ID",
    "DETAIL": {
      "id": "",
      "question": "xxxxxx",
      "answer": "你好吗?",
      "category": "ROOT"
    }
  }
]
```

3.21 删除知识点

URL	/api/faq/del/<faq_id>
HTTP Method	Get
功能	用于删除<faq_id>所指的知识点。(直接依附在 QUESTIONNAIRE_STEPPING、QUESTIONNAIRE_BRANCHING、DRILLDOWN 的 FAQ 不能删除，若要删除需直接删除分类)
输出	若成功，返回被删除知识点 JSON 数据。 【范例】 <pre>{ "MESSAGE": { "id": "111", "categoryid": 24, "question": "111", "answer": "111", "attachments": [] }, "STATUS": true }</pre> 若<faq_id>不存在，系统会返回如下信息。

	【范例】 <pre>{ "MESSAGE": "FAQ_NOT_FOUND", "STATUS": false }</pre>
--	---

3.22 更新知识点

URL	/api/faq/update
HTTP Method	Post
功能	按规定的 JSON 格式, 对知识库进行处理更新。
输入	数据类型: JSON 对象 - id: 要被更改的知识点的 faq_id, 需要在知识库中存在, 并在引擎中是唯一值。 - question: 可改成任何 UTF8 文字, 是知识点的标准问。 - answer: 可改成任何 UTF8 文字, 是知识点的标准答。 - category(optional): 任何 UTF-8 文字, 是知识库中的分类。支持以下三种格式: • 分类路径 (Category Path) , 各层分类名称之间用“->”来分隔; • 分类 ID (Category ID) ; • 分类名称 (Category Name) : 如果分类名称在系统中不唯一, 则分类名称不适用。 【范例】 <pre>{"id":"wether0004","question":"wether0004","answer":"wether0004","category":"ROOT"}</pre>
输出	数据类型: JSON 对象 【范例】 <pre>{ "STATUS": true, "MESSAGE": { "id": "wether0004", "question": "wether0004", "answer": "wether0004", "type": "faq", "sort": 9999, "categoryid": 1 } }</pre>

修改不存在的 faq, 返回信息:

```
{
  "STATUS": false,
  "MESSAGE": "FAQ_NOT_FOUND"
}
```

3.23 新增知识点附件

URL	/api/faq/attach/<faq_id>
HTTP Method	Post
功能	用于上传附件到知识库保存, <faq_id>是指示系统上传附件档案于那一个知识点内。
输入	数据类型: File (form-data name="data") 可上传任何类型文件到系统保存, 但附件大小建议不能超过 5MB
输出	成功输出: <pre>{ "MESSAGE": 201, "STATUS": true, "filename": "201983162630p49t0_0_20190803-162630-default_rid-00132576.wav" }</pre> 失败输出: <pre>{ "MESSAGE": "FILE_EMPTY", "STATUS": false }</pre> 或 (重复上传同一文件) <pre>{ "MESSAGE": "CANNOT_UPLOAD", "STATUS": false, "filename": "201983162630p49t0_0_20190803-162630-default_rid-00132576.wav" }</pre>

3.24 删除知识点附件

URL	/api/faq/attachment/delete/<attachment_id>
HTTP Method	Get
功能	用于删除知识库中的附件, <attachment_id>是指附件在系统上的唯一编码, 可以从提取知识点功能时获取
输出	若成功, 返回附件的 ID。 <pre>{ "MESSAGE": 203, "STATUS": true }</pre>

3.25 获取知识点附件

URL	/api/faq/attachment/<attachment_id>
HTTP Method	Get
功能	用于获取知识点附件, <attachment_id>是指附件在系统上的唯一编码, 可以从提取知识点功能时获取
输出	数据类型: 文件流 附件文件名可从 HTTP HEADER 中的 Content-Disposition 中获取, 例子如下: Content-Disposition: attachment; filename="<filename>"

3.26 批量更新知识库

URL	/api/faq/import/csv
HTTP Method	Post
功能	将若干条更新记录, 按规定的 CSV 格式 (参照【 知识库记录格式 】) 编写, 然后对知识库进行批量处理更新。
输入	数据类型: File (form-data name="data") 文件格式: CSV

输出	数据类型：JSON 数组 返回已添加的知识库信息。
-----------	------------------------------

【知识库记录格式】

知识库导入有两个部份, 为知识点部分和分类部分, 以下为导入知识点的例子:

ID	ACTION	CATEGORY	QUESTION	ANSWER
###..... 知识点 ID 必须唯一	ADD: 添加 UPDATE: 更新 DELETE: 删除 VERIFY: 验证	"CAT1->CAT2->CAT" (以英文双括号包裹; 上下级分类用->分隔)	"任何文字" (以英文双括号包裹)	"任何文字" (以英文双括号包裹)
例子:				
ID	ACTION	CATEGORY	QUESTION	ANSWER
111	ADD	"个人网银->密码"	"个人网银有哪些密码?"	"四种: 登陆密码、交易密码、短信验证码、证书密码"

第二部分为导入分类资料。该部分不建议改动, 除非操作人员对不同类型分类的使用方法特别清楚。

MODE	ACTION	CATEGORY PATH	DESC	FAQID
分类有五种模式: STANDARD; ENUM; QUESTIONNAIRE_STEPPING; QUESTIONNAIRE_BRANCHING; DRILLDOWN	-CAT: 添加分类	"CAT1->CAT2->CAT" (以英文双括号包裹; 上下级分类用->分隔), 有扩展分类的用" "分隔, 例如: ROOT->POS->产品介绍->AN05 ROOT->collection->FAQs->AN05	"任何文字" (以英文双括号包裹)	"任何文字" (但必须在系统是惟一, 而且 FAQID 所指的知识点必须存在档案内一起创建。 DRILLDOWN, QUESTIONNAIRE_STEPPING, QUESTIONNAIRE_BRANCHING 必须要有 FAQID, 其他分类则可为 null)
例子:				
ID	ACTION	CATEGORY	QUESTION	ANSWER
DRILLDOWN	CAT	"ROOT"	"Special FAQ Question for Drilldown"	"Special FAQ Question for Drilldown"

注意: 在知识点中, 如果该分类不存在, 默认情况下, 系统会自动添加为标准型分类。

【CSV 范本一 更新知识点】(以系统实际导出的数据格式为准)

ID,ACTION,CATEGORY,QUESTION,ANSWER

111,ADD,ROOT->Bank,"How many types of password there?","Three:Login Password, SMS Password and Certificate Password."

112,UPDATE,ROOT->Bank,"The question need to update","The answer need to update"

113,DELETE,ROOT->BANK, "The question need to delete" ,"The answer need to delete"

重要提醒

CSV 的第一行必须是: ID,ACTION,CATEGORY,QUESTION,ANSWER

【CSV 范本二 添加特别分类及知识点】(以系统实际导出的数据格式为准)

ID,ACTION,CATEGORY,QUESTION,ANSWER

111,ADD,ROOT->Bank,"How many types of password there?", "Three: Login Password, SMS Password and Certificate Password."

"QUESTIONNAIRE_STEPPING_CAT",ADD,"ROOT","Special FAQ Question for Questionnaire Stepping Here","Special FAQ Answer for Questionnaire Stepping Here"

"QUESTIONNAIRE_BRANCHING_CAT",ADD,"ROOT","Special FAQ Question for Questionnaire Branching Here","Special FAQ Answer for Questionnaire Branching Here"

"DRILLDOWN_CAT",ADD,"ROOT","Special FAQ Question for Drilldown Here","Special FAQ Question for Drilldown Here"

#START OF CATEGORY

#MODE,ACTION,PATH,DESC,FAQID

STANDARD,CAT,"ROOT->STANDARD_CAT","Standard Desc",

ENUM,CAT,"ROOT->ENUM_CAT","Enum Desc",

QUESTIONNAIRE_STEPPING,CAT,"ROOT->QUESTIONNAIRE_STEPPING_CAT","Questionnaire Stepping Desc","QUESTIONNAIRE_STEPPING_CAT"

QUESTIONNAIRE_BRANCHING,CAT,"ROOT->QUESTIONNAIRE_BRANCHING_CAT","Questionnaire Branching Desc","QUESTIONNAIRE_BRANCHING_CAT"

DRILLDOWN,CAT,"ROOT->DRILLDOWN_CAT","Drilldown Desc","DRILLDOWN_CAT"

3.27 导出知识库

URL	/api/export/faq
HTTP Method	Get
功能	从引擎下载知识库。
输出	CSV 格式（详见 批量更新知识库 中的 【知识库记录格式】 ）文件，可作本地存储。

3.28 重置知识库

URL	/api/faq/clear
HTTP Method	Get

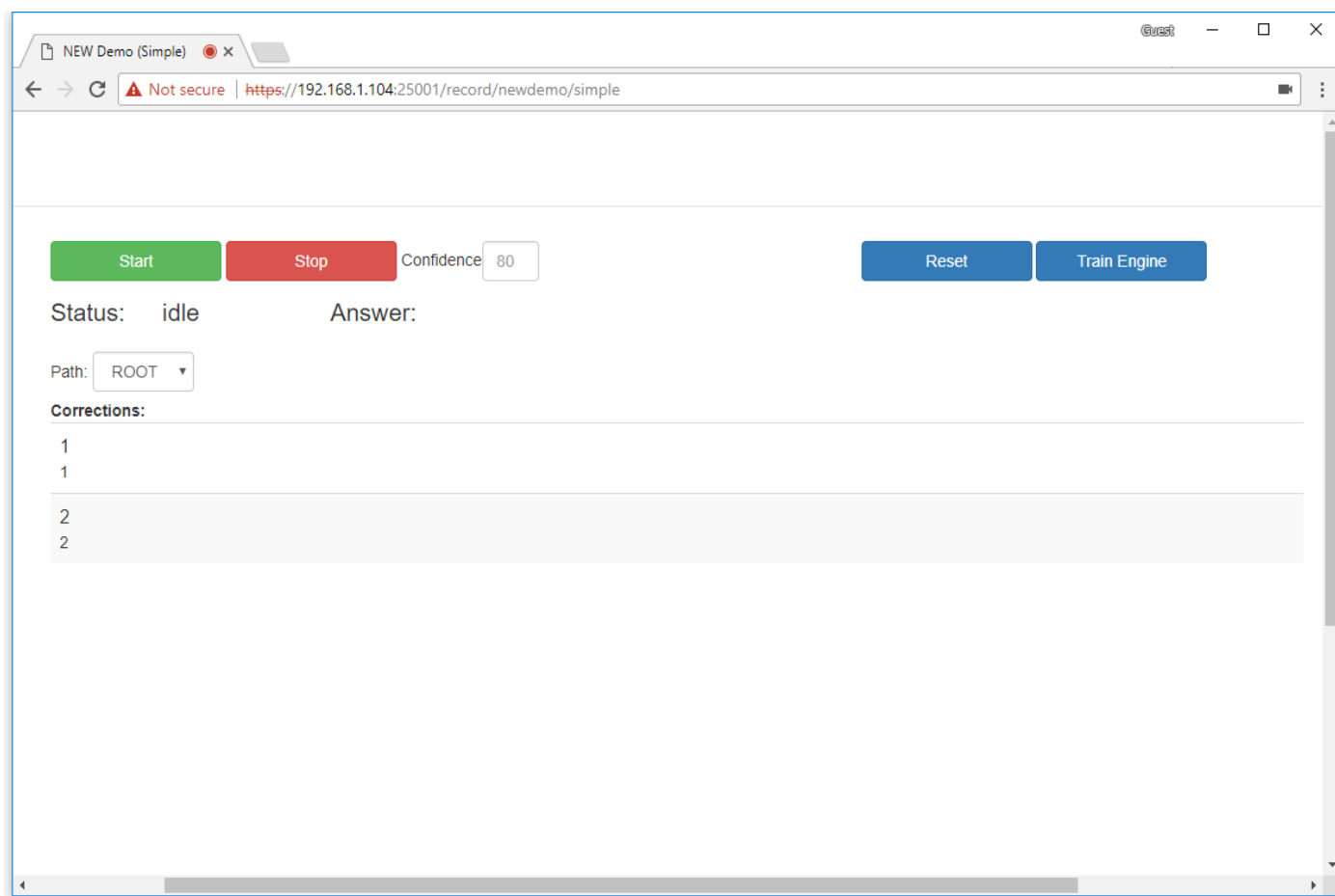
功能	将整个知识库重置为初始状态（全空）。重置后不能恢复, 请用户慎用此功能。
输出	若成功, 返回 {"STATUS": "OK"}。

四、人机交互记录管理

业务人员通过『人机交互记录管理』这个工具, 可查看用户与 NLP Engine 的所有沟通记录, 并可对引擎给用户的答复进行高效的认可或纠正。认可和纠正所产生的数据将用于引擎的机器自学习。

前面的API只是增加了知识库中的知识点, 若要引擎自动懂应答问题, 还需要每一个知识点都有训练数据去训练引擎。有几个地方可以增加知识点的训练数据, 其中一个就是利用/chat.jsp 和/report.jsp结合运用, 增加训练数据。在这里要注意一点,/report.jsp是须要至少有ROLE_USER权限才可以进行操作。

开发者可通过访问/chat.jsp和/report.jsp, 自助产生『人机交互记录』, 如下两图所示:



GateAdmin

 Add Users
 Import/Export
 Data Tagging
 Data Review
 Data Allocation
 Response

Role Permissions
 Tree Editor
 Replace
 Import/Export
 Response
 Workstation
 Data Statistics
 Sub Functions
 Audit Trail
 Data Allocation
 Data Review
 Data Tagging

Create Date:
 Category Path:
 Name:
 Node ID:
 Expected Node:
 CL%:
 Query ID:

☐	ID	Timestamp	Category Path	Name	Node ID	Expected Node ID	CL%	Qu
☐	6551	2020-11-25 15:43:48	ROOT->SuperBankDemo->ZhMenu	Download Play	CNA1		32.08	20201125-133
☐	6550	2020-11-25 15:43:48	ROOT->SuperBankDemo->ZhMenu	Download Play	CNA1		28.78	20201125-133
☐	6549	2020-11-25 15:43:48	ROOT->SuperBankDemo->ZhMenu	Download Play	CNA1		28.78	20201125-133
☐	6548	2020-11-25 15:43:48	-	Download Play	AN02		0.00	83ece1eb-2c4a-4bc
☐	6547	2020-11-25 15:43:48	-	Download Play	AN02		0.00	16042892

/report.jsp内, 引擎会列出所有相关人员在/chat.jsp询问过的或者通过CSV文件导入的“自然问题”。所有人机交互数据都可以用作机器学习。用户也可按条件搜索相应的交互记录。

以下是管理训练数据 API 的使用规范:

4.1 搜索人机交互记录

URL	/api/response/search
HTTP Method	Post
功能	根据定制条件, 搜索人机交互记录。所有搜索条件均为选填。
输入	数据类型: JSON 对象 - from: 起始时间, 人机交互记录的更新时间。(MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss) - to: 截止时间, 人机交互记录的更新时间。(MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss) - createFrom: 起始时间, 人机交互记录的创建时间。(MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss) - createTo: 截止时间, 人机交互记录的创建时间。(MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss) - minConfidence: 最低自信值 (0.01-1.0) - maxConfidence: 最高自信值 (0.01-1.0) - includeTraining: 是否包括初始训练数据 true/false (即: faqid=0)

	- mismatch: 是否包括 faqid 和 expectedFaqid 不一致的训练数据, true/false - responseld: 用户使用自然语言表达的问题的唯一标识号, 大小写敏感。 - question: 任何 UTF8 文字, 是用户提供给引擎训练的自然语言表达的问题。 - questionId: questionId 有三种格式: • 训练数据对应的分类路径, 分类路径中的各分类以“->”间隔。例如: “ROOT->A->B”。在分类路径后添加“%”, 系统会用“%”前的分类路径进行模糊查询。例如 “ROOT->A->B%”。 • 训练数据对应的分类 ID。 • 训练数据对应的分类名称。如果分类名称在系统中不唯一, 则分类名称不适用。 - faqid: 知识库中存在的知识点的 ID, 引擎中的唯一值。 - expectedFaqid: 对应用问题的知识点 (意图) ID。请注意大小写区分。 - pageSize: 对搜索记录进行分页的每页记录条数。 - page: 对搜索记录进行分页的页码, 只在 pageSize 有效时使用。 【范例】 <pre>{ "from": "11/23/2017'T'12:20:20", "to": "12/29/2017'T'12:20:20", "createFrom": "11/23/2017", "createTo": "12/29/2017", "minConfidence": "0.01", "maxConfidence": "1.0", "includeTraining": true, "mismatch": false, "responseld": "CHAT_1511958003305", "question": "你好", "questionId": "ROOT->A", "faqid": "1", "expectedFaqid": "1", "pageSize": "5", "page": "3" }</pre>
输出	数据类型: JSON 数组 【范例】 <pre>[{ "id": 23, "responseld": "CHAT_1511958003305", "questionId": "default", "question": "123",</pre>


```
"faqid": "3",  
"expectedFaqid": null,  
"responder": null,  
"user": "user_account",  
"userip": "192.168.1.115",  
"createTimestamp": 1511958002000,  
"timestamp": 1511958003000,  
"confidence": 20  
}  
]
```

4.2 按搜索条件对人机交互记录进行计数

URL	/api/response/search/count
HTTP Method	Post
功能	根据定制条件，搜索人机交互记录的条数。该功能用于对 搜索人机交互记录 中的交互记录进行分页。所有搜索条件均为选填。
输入	数据类型：JSON 对象 - from: 起始时间，人机交互记录的更新时间。（MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss） - to: 截止时间，人机交互记录的更新时间。（MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss） - createFrom: 起始时间，人机交互记录的创建时间。（MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss） - createTo: 截止时间，人机交互记录的创建时间。（MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss） - minConfidence: 最低自信值（0.01-1.0） - maxConfidence: 最高自信值（0.01-1.0） - includeTraining: 是否包括初始训练数据 true/false（即：faqid=0） - mismatch: 是否包括 faqid 和 expectedFaqid 不一致的训练数据，true/false - responseld: 用户使用自然语言表达的问题的唯一标识号，大小写敏感。 - question: 任何 UTF8 文字，是用户提供给引擎训练的自然语言表达的问题。 - questionId: questionId 有三种格式： • 训练数据对应的分类路径，分类路径中的各分类以“->”间隔。例如：“ROOT->A->B”。在分类路径后添加“%”，系统会用“%”前的分类路径进行模糊查询。例如“ROOT->A->B%”。 • 训练数据对应的分类 ID。 • 训练数据对应的分类名称。如果分类名称在系统中不唯一，则分类名称不适用。 - faqid: 知识库中存在的知识点的 ID，引擎中的唯一值。 - expectedFaqid: 对应用户问题的知识点（意图）ID。请注意大小写区分。 【范例】 <pre>{ "from": "11/23/2017'T'12:20:20",</pre>

	<pre>"to": "12/29/2017'T'12:20:20", "createFrom": "11/23/2017", "createTo": "12/29/2017", "minConfidence": "0.01", "maxConfidence": "1.0", "includeTraining": true, "mismatch": false, "responseld": "CHAT_1511958003305", "question": "你好", "questionId": "ROOT->A", "faqid": "1", "expectedFaqid": "1", }</pre>
输出	数据类型：JSON 对象 - total: 按搜索条件搜索出的人机交互记录条数。开发者根据该参数自行计算每页记录条数和分页页数，并在搜索人机交互记录中指明。 【范例】 <pre>{ "total": 8 }</pre>

4.3 更新人机交互记录中的“expectedFaqid”

URL	/api/response/update/expectedFaqid (原/response/update/expectedFaqid)
HTTP Method	Post
功能	更新人机交互记录中的“expectedFaqid”，用于机器学习。
输入	数据类型：JSON 数组 使用 Array of hash（哈希数组）可批处理多条记录。 - responseld: 人机交互记录 ID（必须唯一） - expectedFaqid: 对应用户问题的知识记录（标准问题） ID 【范例】 <pre>[{"responseld": "1133", "expectedFaqid": "111"}, {"responseld": "1134", "expectedFaqid": "234"}]</pre>
输出	数据类型：JSON 数组 若成功，返回{"MESSAGE": [], "STATUS": "OK"}；若不成功，返回错误信息。 【范例】 <pre>{ "MESSAGE": [], "STATUS": "OK" }</pre>

请注意，在人机交互记录中填入“expectedFaqid”是对引擎进行训练时必不可少的步骤。在操作中，开发者也可以用通过/report.jsp来更新人机交互记录中的“expectedFaqid”，如下图所示。

FAQ

- FAQ编辑器

- 导入/导出

聊天

响应

- 报告

- 导入/导出

用户

配置

Update Date From: To:

QID: Question: Faqid: Expected Faqid: Responseld:

Create Date From: To:

Confidence Min: Max:

☐ 包含训练数据 ☐ FAQID错误匹配 [重置](#) [搜索](#) [训练引擎](#)

«
1
2
3
»
 Go

ID	Timestamp	QID	Question	Faqid	Expected Faqid	CL%	Responseld
17	22-11-2017 19:23:19	default	Download Play	-1		0.00	2017_11_22_19_23_15_3324
18	22-11-2017 19:23:28	default	Download Play	-1		0.00	2017_11_22_19_23_25_3569
19	22-11-2017 19:23:38	default	Download Play	-1		0.00	2017_11_22_19_23_34_8293
20	22-11-2017 19:23:45	default	Download Play	-1		0.00	2017_11_22_19_23_41_1172
21	22-11-2017 19:24:08	ROOT->D1	Download Play	-1		0.00	2017_11_22_19_24_4_2686
22	22-11-2017 19:24:17	ROOT->D1	Download Play	-1		0.00	2017_11_22_19_24_13_2822
23	22-11-2017 19:24:44	ROOT->D2	Download Play	-1		0.00	2017_11_22_19_24_40_928
24	22-11-2017 19:24:56	ROOT->D2	Download Play	-1		0.00	2017_11_22_19_24_49_6306
25	22-11-2017 19:27:37	default	Download Play	1		51.28	2017_11_22_19_27_31_4843
26	22-11-2017 19:27:48	default	Download Play	2		43.24	2017_11_22_19_27_41_8767
27	22-11-2017 19:28:04	default	Download Play	1		47.90	2017_11_22_19_27_57_4934
28	22-11-2017 19:31:05	ROOT->D1	Download Play	2		74.04	001
29	22-11-2017 19:32:25	ROOT->D1	Download Play	2		74.04	002
30	22-11-2017 19:32:55	ROOT->D2	Download Play	4		98.22	003
31	22-11-2017 19:33:27	ROOT->D1	Download Play	1		72.98	2017_11_22_19_33_13_6149
32	22-11-2017 19:34:25	ROOT->D2	Download Play	3		90.06	004
33	23-11-2017 15:03:01	ROOT->D1	Download Play	2		79.21	005
34	23-11-2017 16:23:49	ROOT->D1	Download Play	2		74.04	006
35	28-11-2017 15:46:00	default	Download Play	3		5.00	23a50b4e-9cb5-44be-9ed1-cc
36	28-11-2017 17:43:38	default	Download Play	3		5.00	bb4535ff-43fb-4d90-86af-8ee

1/你好吗?
 STEPPING_1_FAQ/STEPPING_1_Q
 BRANCHING_1_FAQ/BRANCHING_1_Q
 DRILLDOWN_1_FAQ/DRILLDOWN_1_Q

4.4 按“expectedFaqid”删除人机交互记录

URL	/api/response/del/faqid/<expectedFaqid>
HTTP Method	Get
功能	按指定的“expectedFaqid”删除对应的人机交互记录。该功能用于删除系统中不存在的知识点所对应的人机交互记录。
输入	expectedFaqid: 对应用户问题的知识记录 ID。待删除的人机交互记录中的“expectedFaqid”。开发者需要指明确切的“expectedFaqid”或者指明“expectedFaqid”的前缀中至少首 4 个字符。 注意： 如果输入值不是确切的“expectedFaqid”，系统会根据输入的字符搜索所有符合的“expectedFaqid”，然后删除所有对应的人机交互记录，而不再进行确认。
输出	数据类型：JSON 对象 - MESSAGE: 返回该指令的执行消息，有以下三种情况： <数值>: 删除的人机交互记录。(status 为 true) “PARAMETER_TOO_SHORT”: 输入的字符少于 4 个。(status 为 false) “NULL_OUTPUT”: 输入的字符匹配不到任何 FAQ。(status 为 false) - status: 该指令的执行结果状态，true 或 false。 【范例】 <pre>{ "MESSAGE": 2, "STATUS": true }</pre>

4.5 同步人机交互记录（仅适用于多机版引擎）

URL	/api/schedule/ml/remoteSyncJob/<responseDate>
HTTP Method	Get
功能	将所有 RTSA 服务器中的人机交互记录(包括相关语音)同步到 ML 服务器，其中 rtas 的“expectedFaqid”不会覆盖到 ML，所有的训练数据标注都在 ml 进行。 注意：该指令需要在 ML 服务器执行。仅提交同步请求，不包含同步状态。ml 引擎默认每隔 15 分钟从各 rtas 同步一次数据，间隔时间可设置。
输入	responseDate: ml 从各 rtas 获取>=<responseDate>的 response 数据。日期格式为 yyyymmdd，例如：20210331。若输入 00000000，默认使用引擎记录的最后一次同

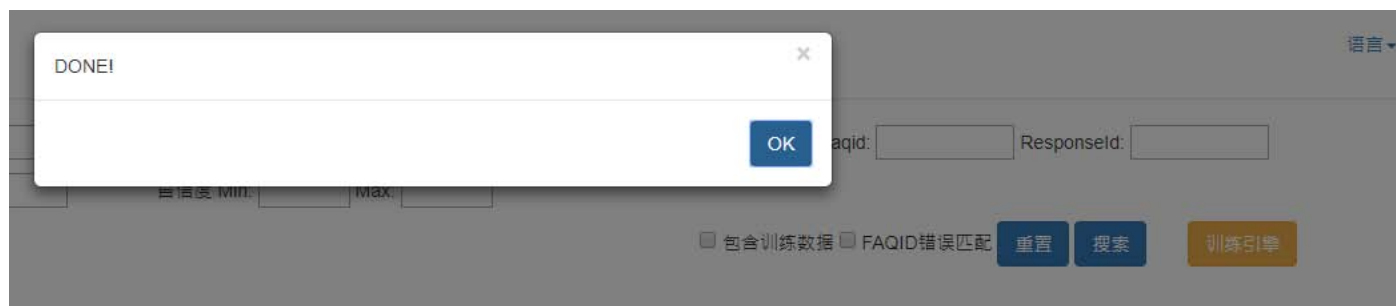
	步时间，记录的时间精确到 yyyymmdd:hh:mm:ss。
输出	数据类型：JSON 对象 - STATUS: 返回该指令的提交状态，true 或 false。 【范例】 <pre>{ "STATUS": true }</pre>

五、 机器自学习

业务人员在处理完所有的「人机交互记录」（认可或纠正 NLP Engine 的回复）后，可一键触发机器自学习。如下图所示：



机器自学习每次所需时间视数据量而定。请耐心等待，直至弹出下图信息对话框，表示机器自学习完毕。



以下是该类 API 的使用规范：

5.1 批量导出训练数据

URL	/api/report/csv/export
HTTP Method	Get
功能	从引擎导出全部报表数据。只有 faqid 及 expectedFaqid 都要有数值的报表数据才会被导出。
输出	数据类型: File 文件格式: CSV <pre> "#timestamp" responseld faqid expectedFaqid confidence questionId question <timestamp_1> <responseld_1> <faqid_1> <expectedFaqid_1> <confidence_1> <questionid_1> <question_1> <timestamp_2> <responseld_2> <faqid_2> <expectedFaqid_2> <confidence_2> <questionid_2> <question_2> </pre> 注意: 每个字段之间用一个制表符<Tab>键间隔, 而不是空格键。 - timestamp: 该记录的创建时间。 - responseld 该记录在引擎里的唯一编码。 - faqid: 引擎回答这个自然问题的知识点(意图) ID。 - expectedFaqid: 对应这个自然问题的正确知识点(意图)ID。 - confidence: 引擎选取答案的自信值 (0.0 – 100.0) 。 - questionId: 该记录对应的分类路径。 - question: 该记录引擎信息。 【报表数据 CSV 范例】 <pre> "#timestamp" responseld faqid expectedFaqid confidence questionId question 2015-12-02T14:59:05 1449039538130 1 1 100.0 default SIL cMan TeMan 2015-12-02T15:00:10 1449039603210 1 1 100.0 default Weng AHeng n SIL </pre>

5.2 筛选导出训练数据

URL	/api/report/csv/export
HTTP Method	POST
功能	从引擎导出全部报表数据。只有 faqid 及 expectedFaqid 都要有数值的报表数据才会被导出。
输入	数据类型: JSON 对象 使用 Array of hash (哈希数组) 可批处理多条记录。 【报表数据搜寻格式】 <pre>{ "responseld": "<responseld>", "minConfidence": <minConfidence>, "maxConfidence": <maxConfidence>, "faqid": "<faqid>", "expectedFaqid": "<expectedFaqid>", "from": "<DateTime>", "to": "<DateTime>", "questionId": "<questionId>" }</pre> - responseld: 用户使用自然语言表达的问题的唯一标识号。请注意大小写区分。responseld 可作局部匹配, 例如在数据库中有一条训练数据的 responseld 是 123456789, 只要在这个 responseld 搜寻参数用上 4567, 系统也会将这个 responseld 的搜寻出来。例如要搜出所有报表数据当中 responseld 有 4567, 可作以下的 JSON 筛选条件输入: {"responseld": "4567"} - minConfidence: 最低自信值。是一个由 0-100 的数值。如用户想搜寻所有报表数据自信值不低于 50, 可作以下的 JSON 筛选条件输入: {"minConfidence": 50} - maxConfidence: 最高自信值。是一个由 0-100 的数值。如用户想搜寻所有报表数据自信值不高于 50, 可作以下的 JSON 筛选条件输入: {"maxConfidence": 50} - faqid: 在这一条报表数据中, 引擎所选择的意图。这个必须要完全匹配(exact match)知识库中的知识点 ID。 - expectedFaqid: 在这一条报表数据中, 用户或静默坐席所选择的意图。这个必须要完全匹配(exact match)知识库中的知识点 ID。 - from: 起始时间, 人机交互记录的更新时间。(MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss) - to: 截止时间, 人机交互记录的更新时间。(MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss)

	DD'T'hh:mm:ss) - createFrom: 起始时间, 人机交互记录的创建时间。(MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss) - createTo: 截止时间, 人机交互记录的创建时间。(MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss) - questionId: questionId 有三种格式: • 训练数据对应的分类路径, 分类路径中的各分类以“->”间隔。例如: “ROOT->A->B”。在分类路径后添加“%”, 系统会用“%”前的分类路径进行模糊查询。例如 “ROOT->A->B%”。 • 训练数据对应的分类 ID。 • 训练数据对应的分类名称。如果分类名称在系统中不唯一, 则分类名称不适用。 以上的筛选条件, 可作单一或组合式筛选条件, 例如用户想导出在 2017 年 2 月 1 日至 2017 年 2 月 2 日 faqid 及 expectedFaqid 为 CHECKACCT, questionId 为 ACCT, 自信值在 50 至 80 之间, responseld 有出现“abc4567”值的报表纪录, 用户可组合出以下筛选条件: 【范例】 <pre>{ "responseld": "abc4567", "minConfidence": 50, "maxConfidence": 80, "faqid": "CHECKACCT", "expectedFaqid": "CHECKACCT", "from": "2017-02-01T00:00:00", "to": "2017-02-02T23:59:59", "questionId": "ACCT" }</pre>
输出	数据类型: File 文件格式: CSV 【报表数据 CSV 格式】 <pre>"#timestamp" responseld faqid expectedFaqid confidence questionId question <timestamp_1> <responseld_1> <faqid_1> <expectedFaqid_1> <confidence_1> <questionid_1> <question_1> <timestamp_2> <responseld_2> <faqid_2> <expectedFaqid_2> <confidence_2> <questionid_2> <question_2></pre> 注意: 每个字段之间用一个制表符<Tab>键间隔, 而不是空格键。 - timestamp: 该记录的创建时间。

	- responseId 该记录在引擎里的唯一编码。 - faqid: 引擎回答这个自然问题的知识点(意图) ID。 - expectedFaqid: 对应这个自然问题的正确知识点(意图)ID。 - confidence: 引擎选取答案的自信值 (0.0 – 100.0) 。 - questionId: 该记录对应的分类路径。 - question: 该记录引擎信息。 【报表数据 CSV 范例】 <pre> "#timestamp" responseId faqid expectedFaqid confidence questionId question 2015-12-02T14:59:05 1449039538130 1 1 100.0 default SIL cMan TeMan 2015-12-02T15:00:10 1449039603210 1 1 100.0 default Weng AHeng n SIL </pre>
--	--

5.3 批量导入训练数据(不含语音文件)

URL	/api/report/csv/import (原 API: /train/csv)
HTTP Method	Post
功能	按规定的 CSV 格式（参照以下的 【训练数据 CSV 格式】 ）批量编写训练数据，上传至引擎，不含语音文件。需要注意：1、知识库中不存在 faqid 的训练数据不会导入引擎；2、在引擎已存在的训练数据，会被覆盖；3、没有的训练数据会新增。
输入	数据类型: File (form-data name="data") 文件格式: CSV 【训练数据 CSV 格式】 <pre> "#timestamp" responseId faqid expectedFaqid confidence questionId question <timestamp_1> <responseId_1> <faqid_1> <expectedFaqid_1> <confidence_1> <questionid_1> <question_1> <timestamp_2> <responseId_2> <faqid_2> <expectedFaqid_2> <confidence_2> <questionid_2> <question_2> </pre> 注意：每个字段之间用一个制表符<Tab>键间隔，而不是空格键。 【训练数据 CSV 范例 1】 <pre> "#timestamp" responseId faqid expectedFaqid confidence questionId question 02-12-2015 14:59:05 1449039538130 1 1 100.0 default Hello 02-12-2015 15:00:10 1449039603210 1 1 100.0 default How are you </pre>

	【训练数据 CSV 范例 2】 <table><tr><th>#timestamp</th><th>responseld</th><th>faqid</th><th>expectedFaqid</th><th>confidence</th><th>questionId</th><th>question</th></tr><tr><td>2015-12-02T14:59:05</td><td>1449039538130</td><td>1</td><td>1</td><td>100.0</td><td>default</td><td>Hello</td></tr><tr><td>2015-12-02T15:00:10</td><td>1449039603210</td><td>1</td><td>1</td><td>100.0</td><td>default</td><td>How are you</td></tr><tr><td>.....</td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	#timestamp	responseld	faqid	expectedFaqid	confidence	questionId	question	2015-12-02T14:59:05	1449039538130	1	1	100.0	default	Hello	2015-12-02T15:00:10	1449039603210	1	1	100.0	default	How are you							
#timestamp	responseld	faqid	expectedFaqid	confidence	questionId	question																							
2015-12-02T14:59:05	1449039538130	1	1	100.0	default	Hello																							
2015-12-02T15:00:10	1449039603210	1	1	100.0	default	How are you																							
.....																													
输出	若执行成功，引擎将返回提交成功的信息。 【范例】 <pre>{ "STATUS": "SUBMIT" }</pre>																												

5.4 重置训练数据库

URL	/api/response/clear
HTTP Method	Get
功能	将整个训练数据库重置为初始状态（全空）。数据库重置后不能恢复, 请用户慎用此功能。
输出	若成功, 返回 {"STATUS": "OK"}。

5.5 触发引擎自学习

URL	/train/db
HTTP Method	Get
功能	使用引擎里的全部训练数据, 触发引擎自学习, 即全部训练。
输出	数据类型: JSON 对象 - STATUS: 返回引擎自学习状态。“SUBMIT”表示成功触发引擎自学习; “BUSY”表示引擎正在进行自学习。 【范例】 {"STATUS": "SUBMIT"}

5.6 触发针对指定分类的引擎自学习

URL	/train/db
HTTP Method	Post
功能	使用引擎里指定分类下的训练数据，触发引擎自学习，即部分训练。而未指定的分类下的训练数据则不会被训练。
输入	数据类型：JSON 对象 - engineId: 为需要训练的分类路径，分类路径中的各个分类之间用“->”间隔。每次可以指定一个或多个分类路径进行训练。 【范例】 <pre>{"engineId": ["ROOT->A", "ROOT->B->C"]}</pre>
输出	数据类型：JSON 对象 - STATUS: 返回引擎自学习状态。“SUBMIT”表示成功触发引擎自学习；“BUSY”表示引擎正在进行自学习。 【范例】 <pre>{"STATUS": "SUBMIT"}</pre>

5.7 查看引擎自学习状态

URL	/train/status
HTTP Method	Get
功能	查看引擎当前的自学习状态。
输出	数据类型：JSON 对象 若引擎自学习完成，引擎将返回： <pre>{ "MESSAGE": null, "NAME": "TrainingHelper", "RESULT": "DONE", "STATUS": "IDLE" }</pre> 若引擎未完成自学习，引擎将返回： <pre>{ "NAME": "TrainingHelper", "STATUS": "RUNNING" }</pre>

5.8 同步引擎（仅适用于多机版引擎）

URL	/api/engine/sync
HTTP Method	Get
功能	将 ML 服务器训练后的数据同步到 RTSA 服务器。 注意：该指令需要在 ML 服务器执行，仅提交同步请求，不包含同步状态。
输出	数据类型：JSON 对象 - MESSAGE: 返回该指令的执行消息。 “BUSY”表示同步引擎正在进行（status 为 false）； “DONE”表示同步引擎已经完成（status 为 true）。 - STATUS: 返回该指令的提交状态，true 或 false。 【范例】 <pre>{ "id": "syncEngine", "MESSAGE": "BUSY", "STATUS": false }</pre>

5.9 查看同步引擎状态（仅适用于多机版引擎）

URL	/api/async/status/syncEngine
HTTP Method	Get
功能	查看同步引擎状态。 注意：该指令需要在 ML 服务器执行。
输出	数据类型：JSON 对象 - MESSAGE: 返回该指令的执行消息。“RUNNING”表示同步引擎正在进行中。 - STATUS: 返回该指令的提交状态，true 或 false。 【范例】 <pre>{ "MESSAGE": "RUNNING", "STATUS": false }</pre>

六、语音意图识别

6.1 上传语音文件进行语音意图识别

URL	/api/uaudio/file/<questionId>/<responseId>
HTTP Method	Post
功能	上传特定格式的语音文件到引擎，引擎会对该语音文件进行意图识别，返回所有可能匹配的知识点列表，按自信值由大到小排列。
输入	File (form-data name="data") 文件类型：WAV 文件，语音格式为 signed 16 bit pcm(raw) mono 格式 - questionId: 支持以下三个格式： • 分类路径 (Category Path)，各层分类名称之间用“->”来分隔； • 分类 ID (Category ID)； • 分类名称 (Category Name)：如果分类名称在系统中不唯一，则分类名称不适用。 - responseId: 用户使用自然语言表达的问题的唯一标识号。需要开发者指定，且大小写敏感。
输出	数据类型：JSON 数组 - responseId: 用户使用自然语言表达的问题的唯一标识号。 - answer: 匹配到的知识点的标准答案。 • 若匹配到标准型或者枚举型分类中的知识点，此值为匹配到的知识点的答案。 • 若匹配到其他分类中的知识点，此值为“followUp”参数中指定的分类中，特殊知识点的答案。 - question: 匹配到的知识点的标准问题。 • 若匹配到标准型或者枚举型分类中的知识点，此值为匹配到的知识点的问题。 • 若匹配到其他分类中的知识点，此值为“followUp”参数中指定的分类中，特殊知识点的问题。 - confidence: 引擎选取答案的自信值 (0.0 – 100.0)。 - followUp: • 若匹配到标准型或者枚举型分类中的知识点，该参数不适用。 • 若匹配到其他分类中的知识点，为匹配到的知识点所属的分类的分类路径。 - y: 匹配到的知识点的标准问题。 • 若匹配到标准型或者枚举型分类中的知识点，此值为匹配到的知识点的问题。 • 若匹配到其他分类中的知识点，此值为“followUp”参数中指定的分类中，特殊知识点

	的问题。 【范例】 <pre>[{ "responseId": "006", "answer": "B", "confidence": "74.04", "question": "B", "followUp": "ROOT->D2", "y": "2" }, { "responseId": "006", "answer": "A", "confidence": "25.96", "question": "A", "y": "1" }]</pre>
--	---

6.2 下载已识别的语音文件

URL	/api/wav/<responseId>
HTTP Method	Get
功能	根据 responseId，从引擎中下载已识别的语音文件。
输入	- responseId：用户使用自然语言表达的问题的唯一标识号。需要开发者指定，且大小写敏感。
输出	WAV 格式的语音文件。 如果 responseId 或者响应的语音文件不存在，引擎将不会返回任何信息。

除了上传语音文件进行语音意图识别之外，引擎还支持通过 websocket 调用引擎接口进行语音意图识别。使用 websocket, URL 入口有两种，一种是 ws, 另一种是加密通道 wss。譬如使用/api/uaudio/websocket 接口, 则 URL 在 ws 情况下是:

- 所有URL前面应加ws://<Server Address>:<Server Port#>, 其中Server_Address和Server_Port#分配给Application Developer独立使用。如被分配加密通道接口, 即在URL前应加wss://<Server Address>:<Server Port#>。

如: /api/uaudio/websocket, 如果被分配的Server_Address为nlp.ts.com, Port#为 12345, 则该URL的完整应为: ws:// nlp.ts.com:12345/api/uaudio/websocket。如被分配加密通道接口, 则该URL的应为: wss:// nlp.ts.com:12345/api/uaudio/websocket。

现在引擎只支持辨识 Signed 16bit PCM(Raw)单声道(Mono)格式的语音流, 采样率为 8000 到 44100 之间。开发者必须了解 websocket 开发 (RFC6455) ,才能明白以下接口的意思。

6.3 开启传送语音流

URL	ws://{host}/api/uaudio/websocket
Request Method	Websocket
功能	开始进行语音意图识别，开启传送语音流。
输入	数据类型: JSON 对象 - ACTION: "START", ACTION 有三种取值，分别为 START, STOP, CANCEL。使用 START 开启 websocket 连接之后，开发者需要向传送语音流，当语音流传送结束，开发者需要自行发送 STOP 或者 CANCEL 关闭 websocket 连接。 注意: 只有之前开启的 Websocket 连接关闭之后，开发者才可以开启新的连接。STOP 和 CANCEL 的具体使用方法，参见 6.4 停止传送语音流 和 6.5 取消传送语音流 部分。 - responseId: 用户使用自然语言表达的问题的唯一标识号。需要开发者指定，且大小写敏感。 - questionId: 支持以下三种格式： • 分类路径 (Category Path) , 各层分类名称之间用“->”来分隔; • 分类 ID (Category ID) ; • 分类名称 (Category Name) : 如果分类名称在系统中不唯一，则分类名称不适用。 注意: 每次开启 Websocket 连接，可以指定一个或多个 questionId。 - sampleRate (选填) : 语音流的采样率，默认值为 8000，只支持 8000 至 44100。

	- Confidence-Threshold (选填) : 置信度阈值, 取值范围 0-1 (小数点后最多保留两位), 选填。引擎识别结果的置信度高于该值时才会被列出。 - N-Best-List-Length (选填) : 最优列表长度, 取值范围为大于 0 的整数, 选填。引擎返回列表中最多列出多少条识别结果。 除此之外, 引擎还提供语音活性检测 (VAD)参数供开发使用, 该部分参数为选填。具体参数的使用方法参见「附件三: 语音活性检测 (VAD) 参数说明」。 【范例 1】 <pre>{ "ACTION": "START", "sampleRate": 44100, "responseId": "1234567", "questionId": "<QUESTION_ID>", "Confidence-Threshold": 0.3, "N-Best-List-Length": 10 }</pre> 【范例 2】 <pre>{ "ACTION": "START", "sampleRate": 44100, "responseId": "1234567", "questionId": "[<QUESTION_ID_1>,<QUESTION_ID_2>]", "Confidence-Threshold": 0.3, "N-Best-List-Length": 10 }</pre>
输出	数据格式: JSON 对象 - STATUS: 在传送语音流的时候, STATUS 必须为"RECORDING", 如 STATUS 为其他状态, 则代表传送语音予引擎的对话结束。如 STATUS 为"FAIL"状态, 则代表不能开始识别。 - ACTION: 在传送语音流给引擎时, ACTION 必须为"START", "STOP"则代表语音传送结束, 引擎会返回识别结果。"CANCEL"则代表取消本次语音识别, 引擎不会返回识别结果。 - HANDLE: 整数值, 如果 HANDLE 小于 0 代表引擎现在繁忙, 不能开始识别。 【范例】 <pre>{ "STATUS": "RECORDING", "ACTION": "START", "HANDLE": 4 }</pre>

6.4 停止传送语音流

URL	ws://{host}/api/uaudio/websocket
Request Method	Websocket
功能	在语音流传送过程中，停止传送语音流，引擎将返回识别结果。该功能需要与 6.3 启动传送语音流 一起使用。
输入	数据格式：JSON 对象 - ACTION: "STOP", ACTION 有三种取值，分别为 START, STOP, CANCEL。要停止传送中的数据流或者语音流传送结束，开发者需要发送 STOP 来停止，引擎会返回识别结果。 【范例】 <pre>{"ACTION": "STOP"}</pre>
输出	数据格式：JSON 对象 - STATUS: 返回引擎当前状态，"true"或"false"。 - ACTION: 值为"STOP"，表示语音识别结束。 - RESULT: 引擎回复识别结果，结果以 JASON 格式输出。 - responseld: 用户使用自然语言表达的问题的唯一标识号。需要开发者指定，且大小写敏感。 - answer: 匹配到的知识点的标准答案。 • 若匹配到标准型或者枚举型分类中的知识点，此值为匹配到的知识点的答案。 • 若匹配到其他分类中的知识点，此值为"followUp"参数中指明的分类中，特殊知识点的答案。 - question: 匹配到的知识点的标准问题。 • 若匹配到标准型或者枚举型分类中的知识点，此值为匹配到的知识点的问题。 • 若匹配到其他分类中的知识点，此值为"followUp"参数中指明的分类中，特殊知识点的问题。 - confidence: 引擎选取答案的自信值 (0.0 – 100.0) 。 - followUp: 系统认为下一个查询所处的分类位置。 • 若匹配到标准型或者枚举型分类中的知识点，该参数不适用。 • 若匹配到其他分类中的知识点，为匹配到的知识点所属的分类的分类路径。 - y: 匹配到的知识点的标准问题。 • 若匹配到标准型或者枚举型分类中的知识点，此值为匹配到的知识点的问题。 • 若匹配到其他分类中的知识点，此值为"followUp"参数中指明的分类中，特殊知识点的问题。 【范例】

```
{
  "STATUS":true,
  "ACTION":"STOP",
  "RESULT": "
  [
    {
      "responseId":"123456",
      "answer":"MANUALSERVICE",
      "confidence":"8.33",
      "question":"MANUALSERVICE",
      "y":"MANUALSERVICE"
    },
    {
      "responseId":"123457",
      "answer":"AVAILABLECREDIT",
      "confidence":"8.33",
      "question":"AVAILABLECREDIT",
      "y":"AVAILABLECREDIT"
    }
  ]
}
```

6.5 取消传送语音流

URL	/api/uaudio/websocket
HTTP Method	Websocket
功能	在语音流传送过程中, 取消传送语音流, 引擎不会返回任何信息。该功能需要与 启动传送语音流 一起使用。
输入	数据类型: JSON 对象 ACTION: "CANCEL", ACTION 有三种取值, 分别为 START, STOP, CANCEL。要取消传送中的数据流, 开发者需要发送 CANCEL 来取消, 引擎不会返回任何信息。 【范例】 { "ACTION": "CANCEL" }
输出	没有任何输出

用 websocket 连接系统作语音意图识别时, 在代码内基本上是以下流程:

1. init websocket connection
2. calling start api
3. keep writing to the stream until receive message until end
4. close connection

Message call back:

```
if ( "START" == action ){  
    if ( handle <= 0 ){  
        alert("Cannot start recording !!!" + handle);  
    }  
}else if ( "STOP" == action ){  
    stop_stream();  
    handleResultList(["RESULT"]);  
}else if ( "WRITE" == action ){  
    if ( "FAIL" == status ){  
        stop_stream();  
        alert("cannot handle process");  
    }  
}  
}
```

在 websocket 的 API 内, 会以 send binary 作语音传送, send text 作指令传送。如传送语音时出错, 引擎会发出 STATUS 为 "FAIL" 的讯息, 如下:

```
{ "STATUS": "FAIL", "ACTION": "WRITE", "MESSAGE": "NOT_START" }
```


如想索取简单利用 websocket 作语音意图识别的 JAVA 代码例子,可连络我们索取相关资料。

七、 备份管理(数据备份过程需暂停使用引擎)

7.1 检查服务状态（仅适用于多机版引擎）

URL	/api/status/service/all
HTTP Method	Get
功能	列出引擎集群模式中的各个服务状态。已经停止的服务不会在结果中列出。 注意：该指令需要在 ML 服务器执行。
输出	数据格式：JSON 对象 【格式】 <pre>{ "server-name-1": { "service-1": "running", "service-2": "running", "service-3": "running" }, "server-name-2": { "service-1": "running", "service-2": "running", "service-3": "running" } }</pre> 【范例】 <pre>{ "ml-1": { "redis": "running", "ua2x-1": "running", "sentinel7000": "running", "x2y": "running", "mysql": "running" }, "rtsa-1": { "redis": "running", "sentinel7000": "running", "x2y": "running", "ua2x3-2": "running", "ua2x3-1": "running", "mysql": "running" }, "rtsa-2": {</pre>

	<pre> "redis": "running", "sentinel7000": "running", "x2y": "running", "ua2x3-2": "running", "ua2x3-1": "running", "mysql": "running" } </pre>
--	--

7.2 请求下载知识库数据

URL	/api/faq/zip/export/<questionId>
HTTP Method	Get
功能	提交请求下载当前引擎内知识库数据(包括 faq 附件)
输入	- questionId: 分类路径 (Category Path) , 各层分类名称之间用“->”来分隔, <questionId>值为 ROOT 表示下载所有知识库数据。
输出	数据格式: JSON 对象 - MESSAGE: 若该指令执行完, 则返回“DONE”。 - STATUS: 若该指令执行成功, 则返回“true”。 【范例】 <pre> { "MESSAGE": "DONE", "STATUS": true } </pre>

7.3 检查下载知识库数据的请求状态

URL	/api/faq/export/status
HTTP Method	Get
功能	列出已经可供下载的知识库数据包。
输出	数据格式: JSON 数组 - TIME: 请求下载知识库数据的处理时间。若处理完成, 该参数不会显示。 - MESSAGE: 返回请求下载知识库数据的处理状态。 - FILENAME: 生成的 ZIP 格式压缩包文件名, 只有请求下载知识库数据处理完成后, 该参

	数才会被列出。 - STATUS: 下载知识库数据成功, 返回 true。 【范例】 <pre>{ "MESSAGE": "DONE", "FILENAME": "faq_1530763552166.zip", "STATUS": true }</pre>
--	---

7.4 下载知识库数据

URL	/api/faq/tmp/zip/{filename}
HTTP Method	Get
功能	根据 检查下载知识库数据的请求状态 中提供的 FILENAME 下载知识库数据包。
输出	数据类型: File 文件格式: ZIP 注意: ZIP 压缩包里包含 CSV 格式的知识库数据和相关的附件。

7.5 导入知识库数据

URL	/api/faq/zip/import
HTTP Method	Post
功能	导入引擎下载的知识库 zip 数据 (不建议超过 25MB), 导入的数据不能与当前引擎存在的数 据有重复。
输入	数据类型: File (form-data name="data") 文件格式: zip
输出	若执行成功, 引擎将返回提交成功的信息。 【范例】 <pre>{ "STATUS": "SUBMIT" }</pre>

7.6 检查导入知识库数据的状态

URL	/api/faq/import/status
HTTP Method	Get
功能	列出已经可供下载的知识库数据包。
输出	数据格式： - MESSAGE: 若该指令执行完，则返回“DONE”。 - STATUS: 若该指令执行成功，则返回“true”。 【范例】 <pre>{ "MESSAGE": "DONE", "STATUS": true }</pre>

7.7 批量导出详细报表数据

URL	/train/csv/export/extended/all
HTTP Method	Get
功能	从引擎导出详细报表数据。
输出	数据类型: File 文件格式: CSV 【详细训练数据 CSV 格式】 <pre>"#timestamp" responseld faqid expectedFaqid confidence questionId question <timestamp_1> <responseld_1> <faqid_1> <expectedFaqid_1> <confidence_1> <questionid_1> <question_1> <timestamp_2> <responseld_2> <faqid_2> <expectedFaqid_2> <confidence_2> <questionid_2> <question_2></pre> 注意: 每个字段之间用一个制表符<Tab>键间隔, 而不是空格键。 - timestamp: 该记录的创建时间。 - responseld 该记录在引擎里的唯一编码。 - faqid: 引擎回答这个自然问题的知识点(意图) ID。 - expectedFaqid: 对应这个自然问题的正确知识点(意图)ID。 - confidence: 引擎选取答案的自信值 (0.0 – 100.0) 。 - questionId: 该记录对应的分类路径。 - question: 用户输入的自然问题。 【详细训练数据 CSV 范例】 <pre>"#timestamp" responseld faqid expectedFaqid confidence questionId question 2015-12-02T14:59:05 1449039538130 1 1 100.0 default Hello 2015-12-02T15:00:10 1449039603210 1 1 100.0 default How are you</pre>

7.8 请求下载 response 数据

URL	/api/response/zip/export
HTTP Method	Post
功能	提交请求下载当前引擎内 response 数据，不输入条件表示下载引擎所有数据。
输入	数据类型：JSON 对象 - from: 起始时间，人机交互记录的更新时间。（MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss） - to: 截止时间，人机交互记录的更新时间。（MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss） - createFrom: 起始时间，人机交互记录的创建时间。（MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss） - createTo: 截止时间，人机交互记录的创建时间。（MM/DD/YYYY 或 YYYY-MM-DD'T'hh:mm:ss） - questionId: 分类路径，分类路径中的各分类以“->”间隔。例如：ROOT->A->B - faqid: 知识库中存在的知识点的 ID，引擎中的唯一值。 - expectedFaqid: 对应用户问题的知识点（意图）ID。请注意大小写区分。 【范例一】 <pre>{ "from": "05/27/2021", "to": "05/27/2021", "questionId": "ROOT->UnicomFZ->产品介绍", "faqid": "AN05", "expectedFaqid": "AN03" }</pre> 【范例二】 <pre>{ "from": "2021-05-27T01:00:00", "to": "2021-05-27T23:00:00" }</pre>
输出	数据格式：JSON 对象 - MESSAGE: 若该指令执行完，则返回“DONE”。 - STATUS: 若该指令执行成功，则返回“true”。 【范例】 <pre>{ "MESSAGE": "DONE", </pre>

```
"STATUS": true  
}
```

7.9 检查下载 response 数据的请求状态

URL	/api/response/export/status
HTTP Method	Get
功能	列出已经可供下载的 response 数据包。
输出	数据格式: JSON 数组 - TIME: 请求下载 response 数据的处理时间。若处理完成, 该参数不会显示。 - MESSAGE: 返回请求下载 response 数据的处理状态。 - FILENAME: 生成的 ZIP 格式压缩包文件名, 只有请求下载 response 数据处理完成后, 该参数才会被列出。 - STATUS: 下载 response 数据成功, 返回 true。 【范例】 <pre>{ "MESSAGE": "DONE", "FILENAME": "rsp_1530763552166.zip", "STATUS": true }</pre>

7.10 下载 response 数据

URL	/api/tmp/zip/{filename}
HTTP Method	Get
功能	根据检查下载 response 数据的请求状态中提供的 FILENAME 下载 response 数据包。
输出	数据类型: File 文件格式: ZIP 注意: ZIP 压缩包里包含 response 数据和对应的语音。

7.11 导入 response 数据

URL	/api/response/zip/import
HTTP Method	Post
功能	导入引擎下载的 response zip 数据 (不建议超过 25MB) , 导入的数据的 ID 不能与当前引擎存在的数据有重复。
输入	数据类型: File (form-data name="data") 文件格式: zip
输出	若执行成功, 引擎将返回提交成功的信息。 【范例】 <pre>{ "STATUS": "SUBMIT" }</pre>

7.12 检查导入 response 数据的状态

URL	/api/response/import/status
HTTP Method	Get
功能	检查导入 response 数据的状态。
输出	数据格式: JSON 数组 - MESSAGE: 若该指令执行完, 则返回"DONE"。

	- STATUS: 若该指令执行成功，则返回“true”。 【范例】 <pre>{ "MESSAGE": "DONE", "STATUS": true }</pre>
--	---

7.13 请求下载所有语音文件（仅适用于多机版引擎）

URL	/api/housekeeping/folder/download
HTTP Method	Post
功能	请求从 ML 服务器下载某时间范围内的所有语音文件压缩包，文件格式为 zip。 注意：该指令需要在 ML 服务器执行。
输入	数据类型：JSON 对象 - from: 起始时间，引擎处理语音文件的时间。（MM/DD/YYYY 或 YYYY-MM-DDThh:mm:ss） - to: 截止时间，引擎处理语音文件的时间。（MM/DD/YYYY 或 YYYY-MM-DDThh:mm:ss） 注意：建议单次请求时间间隔不超过 90 天。 【范例 – MM/DD/YYYY】 <pre>{ "from": "06/27/2018", "to": "06/29/2018" }</pre> 【范例 – YYYY-MM-DDThh:mm:ss】 <pre>{ "from": "2018-06-27T00:00:00", "to": "2018-07-03T00:00:00" }</pre>
输出	数据格式：JSON 对象 - id: 用于下载 zip 压缩包的唯一识别号。若该指令执行失败，不会返回该参数。 - MESSAGE: 该指令执行失败时，返回该指令执行消息。若该指令执行成功，不会返回该参数。执行消息有以下三种情况： • EMPTY_SET: 选择的日期范围内没有语音文件。

	- INVALID_FROM: 起始时间格式错误。 - INVALID_TO: 截止时间格式错误。 - STATUS: 返回该指令的提交状态, true 或 false。 【范例 - 执行成功】 <pre>{ "id": "6e8b004f-35c3-4889-9e36-c88346324562", "STATUS": true }</pre> 【范例 - 执行失败】 <pre>{ "MESSAGE": "EMPTY_SET", "STATUS": false }</pre>
--	--

7.14 列出可供下载的语音文件包（仅适用于多机版引擎）

URL	/api/async/file/list/{prefix}
HTTP Method	Get
功能	根据语音文件名的前缀列出可供下载的语音文件的服务器存储路径。 注意：该指令需要在 ML 服务器执行。可供下载的语音文件格式为 ZIP 压缩包，语音文件名以 "wav_" 开头，后加 7.6 请求下载所有语音文件（仅适用于多机版引擎） 中返回的 id。
输出	数据格式：JSON 数组 【范例】 <pre>["/usr/local/tomcat/temp/async/wav_05684a99-d8de-4fb7-84f1-193bec7f292b.zip", "/usr/local/tomcat/temp/async/wav_6e8b004f-35c3-4889-9e36-c88346324562.zip", "/usr/local/tomcat/temp/async/wav_4fdde1ed-d354-47d3-9610-47c0773982e2.zip", "/usr/local/tomcat/temp/async/wav_b4840d43-16af-492d-b1b4-9bf2fab379f8.zip"]</pre>

7.15 下载语音文件包（仅适用于多机版引擎）

URL	/api/async/file/get/{id}
HTTP Method	Get
功能	根据 7.6 请求下载所有语音文件（仅适用于多机版引擎） 中返回的 id 下载语音文件。 注意：该指令需要在 ML 服务器执行。
输出	数据类型：File 文件格式：ZIP 注意：ZIP 压缩包包含所选时间范围内的所有语音文件，语音文件按照日期和时间分文件夹存放。

7.16 删除语音文件包（仅适用于多机版引擎）

URL	/api/async/file/del/{id}
HTTP Method	Get
功能	根据 7.6 请求下载所有语音文件 返回的 id 删除语音文件。 注意：该指令需要在 ML 服务器执行。
输出	数据格式：JSON 对象 - MESSAGE: 若该指令执行成功，则返回“DONE”。 - STATUS: 若该指令执行成功，则返回“true”。 【范例】 <pre>{ "MESSAGE": "DONE", "STATUS": true }</pre>

八、配置管理

8.1 更新引擎基本配置参数

URL	/api/config/engine/update
HTTP Method	Post
功能	更新引擎当前基本配置参数。所有输入参数均为选填。
输入	数据格式：JSON 对象 - defaultEngine: 引擎是否为默认训练模式，true 或 false； - caseSensitive: 训练数据的内容是否区分大小写，true 或 false； - accentsSensitive: 训练数据是否口音敏感，true 或 false； - phonicSensitive: 训练数据是否拼读敏感，true 或 false； - removeDuplicate: 配置是否删除重复训练数据，true 或 false； - overQuotaNotifyUrl: 识别次数超出配额后的通知链接； - sessionTimeoutLimit: 引擎页面会话超时时间（秒）； - defaultNBest: 默认最优匹配结果数（大于 0 的整数）； - minGap: 最小自信值差异（0-1.0）； - rebalanceThreshold: 数据重新平衡阈值（默认为 0）； - rebalanceGain: 数据重新平衡增益（0-10），不建议修改。 【范例】 <pre>{ engineSettings: { defaultEngine: true, caseSensitive: false, accentsSensitive: false, phonicSensitive: false, removeDuplicate: false, overQuotaNotifyUrl: null, sessionTimeoutLimit: 1800, defaultNBest: 0, minGap: 0.2, rebalanceThreshold: 0, rebalanceGain: 2.5 } }</pre>

输出	数据格式: JSON 对象 - MESSAGE: 返回该指令的执行结果。若该指令执行成功, 则返回"DONE"。 - STATUS: 返回该指令的执行状态。若该指令执行成功, 则返回"true"。 【范例】 <pre>{ "MESSAGE": "DONE", "STATUS": true }</pre>
-----------	--

8.2 获取意图识别使用量

URL	/api/engine/counter
HTTP Method	Get
功能	获取引擎从部署完成到现在, 最近 12 个月内每月的意图识别使用量。
输出	数据格式: JSON 对象 - MESSAGE: 以 JSON 数组形式返回最近 12 个月内, 每月的意图识别使用量; - STATUS: 返回该指令的执行状态。若该指令执行成功, 则返回"true"。 【范例】 <pre>{ "MESSAGE": [{"201901": 1058}, {"201812": 43}, {"201902": 0}], "STATUS": true }</pre>

8.3 获取引擎当前时间

URL	/api/engine/time/utc
HTTP Method	Get
功能	获取引擎当前时间，以 UTC 格式输出。
输出	数据格式：纯文本 YYYY-MM-DDThh:mm:ss 【范例】 2019-01-01T07:11:12

8.4 获取引擎到期时间

URL	/api/engine/expiry/utc
HTTP Method	Get
功能	获取引擎到期时间，以 UTC 格式输出。
输出	数据格式：纯文本 YYYY-MM-DDThh:mm:ss 【范例】 2019-12-31T15:59:59

8.5 获取引擎详细版本

URL	/api/config/version
HTTP Method	Get
功能	获取引擎详细版本信息，供技术支持使用。
输出	数据格式：JSON 对象 【范例】 <pre>{ X2YCommonVersionInfo: "X2YCommon version: 152abf6", NLPInterfaceVersionInfo: "NLPInterface version: 741d89c", X2YEngineVersionInfo: "X2YEngine version: 123731f", TSBaseVersionInfo: "TSBase version: a05f3ab", A2XAdapterVersionInfo: "A2XAdapter version: 268f7ff" }</pre>

8.6 获取引擎操作日志

URL	/api/auditLog																														
HTTP Method	Get																														
功能	获取引擎系统操作信息，供技术支持使用。 <table border="1"> <thead> <tr> <th>actionType</th><th>actionDetail</th></tr> </thead> <tbody> <tr> <td> exportfaqcsv</td><td> success : export faq_1606119339367.csv </td></tr> <tr> <td> exportfaqzip</td><td> success : export faq_1612338846250.zip </td></tr> <tr> <td> exportresponsecsv</td><td> success : export allExtendedResponse.csv </td></tr> <tr> <td> importfaqcsv</td><td> success : import faq_1595842885825.csv </td></tr> <tr> <td> importfaqzip</td><td> success : import faq_1595929669118.zip </td></tr> <tr> <td> importresponsecsv</td><td> success : import allExtendedResponse.csv </td></tr> <tr> <td> importresponsezip</td><td> success : import rsp_1595831734471.zip </td></tr> <tr> <td> train</td><td> ALL </td></tr> <tr> <td> clearfaq</td><td> success : clear all faqs </td></tr> <tr> <td> clearresponse</td><td> success : clear all responses </td></tr> <tr> <td> deletefaq</td><td> success : {"id":"ttd","categoryid":2,"question":"ttd","answer":"ttd","sort":0,"attachments":[]} </td></tr> <tr> <td> addattachment</td><td> success : 开场白.wav saved to faq {"id":"CP00", "categoryid":1,"question":"嗨问","answer":"有什么可以帮到您呢? ", "sort":0,"attachments":[]} </td></tr> <tr> <td> deleteattachment</td><td> success : 开场白.wav deleted from faq {"id":"CP 00","categoryid":1,"question":"嗨问","answer":"有什么可以帮到您呢? ", "sort":0,"attachments": [{"id":1,"filename":"开场白.wav"}]} </td></tr> <tr> <td>.....</td><td></td></tr> </tbody> </table>	actionType	actionDetail	exportfaqcsv	success : export faq_1606119339367.csv	exportfaqzip	success : export faq_1612338846250.zip	exportresponsecsv	success : export allExtendedResponse.csv	importfaqcsv	success : import faq_1595842885825.csv	importfaqzip	success : import faq_1595929669118.zip	importresponsecsv	success : import allExtendedResponse.csv	importresponsezip	success : import rsp_1595831734471.zip	train	ALL	clearfaq	success : clear all faqs	clearresponse	success : clear all responses	deletefaq	success : {"id":"ttd","categoryid":2,"question":"ttd","answer":"ttd","sort":0,"attachments":[]}	addattachment	success : 开场白.wav saved to faq {"id":"CP00", "categoryid":1,"question":"嗨问","answer":"有什么可以帮到您呢? ", "sort":0,"attachments":[]}	deleteattachment	success : 开场白.wav deleted from faq {"id":"CP 00","categoryid":1,"question":"嗨问","answer":"有什么可以帮到您呢? ", "sort":0,"attachments": [{"id":1,"filename":"开场白.wav"}]}		
actionType	actionDetail																														
exportfaqcsv	success : export faq_1606119339367.csv																														
exportfaqzip	success : export faq_1612338846250.zip																														
exportresponsecsv	success : export allExtendedResponse.csv																														
importfaqcsv	success : import faq_1595842885825.csv																														
importfaqzip	success : import faq_1595929669118.zip																														
importresponsecsv	success : import allExtendedResponse.csv																														
importresponsezip	success : import rsp_1595831734471.zip																														
train	ALL																														
clearfaq	success : clear all faqs																														
clearresponse	success : clear all responses																														
deletefaq	success : {"id":"ttd","categoryid":2,"question":"ttd","answer":"ttd","sort":0,"attachments":[]}																														
addattachment	success : 开场白.wav saved to faq {"id":"CP00", "categoryid":1,"question":"嗨问","answer":"有什么可以帮到您呢? ", "sort":0,"attachments":[]}																														
deleteattachment	success : 开场白.wav deleted from faq {"id":"CP 00","categoryid":1,"question":"嗨问","answer":"有什么可以帮到您呢? ", "sort":0,"attachments": [{"id":1,"filename":"开场白.wav"}]}																														
.....																															

输出	数据类型: JSON 数组 【范例】 <pre>[{ "ID":1, "User_IP":"120.230.113.109", "Username":"user_account", "Action_type":"train", "Action_detail":"ALL", "Start_time":"2020-10-23 17:42:54", "End_time":"2020-10-23 17:42:54" }, { "ID":2, "User_IP":"120.230.113.109", "Username":"user_account", "Action_type":"clearresponse", "Action_detail":"success: clear all responses", "Start_time":"2020-10-23 17:43:48", "End_time":"2020-10-23 17:43:48" }]</pre>
----	--

8.7 获取引擎数据最后导入导出日志

URL	/api/lastImportExportLog
HTTP Method	Get
功能	获取引擎数据最后导入导出日志。
输出	数据类型: JSON 数组 【范例】 <pre>[{ "Username":"user_account", "Action_type":"importfaqcsv", "Start_time":"2020-10-23 17:42:54" }, { "Username":"user_account", "Action_type":"importresponsecsv", "Start_time":"2020-10-23 17:43:48" }, { </pre>

	<pre> "Username": "user_account", "Action_type": "exportfaqcsv", "Start_time": "2020-10-25 17:43:48" }, { "Username": "user_account", "Action_type": "exportresponsecsv", "Start_time": "2020-10-25 17:43:48" }] </pre>
--	--

九、高阶引擎功能

9.1 接口安全认证

一般每一个接口都不需要认证, 这个高阶功能是让某些 API 能有安全认证。 开发商首先用户要准备以下的一个文件发给我们:

```

AUTH_TYPE,ANT_URL,ROLE
BASIC,/api/response/clear,ADMIN
BASIC,/api/faq/clear,ADMIN
BASIC,/queryj/**,LOGIN
BASIC,/api/faq/all,LOGIN
BASIC,/category/all,LOGIN
WEB,/Faq.html,LOGIN
WEB,/,LOGIN
WEB,/report.jsp,LOGIN

```

AUTH_TYPE 是指用什么认证方法, 一是 BASIC, 另一种是 WEB。 BASIC 是如果认证错误, 会报告错误讯息; WEB 是如果认证错误, 会跳转到 LOGIN 页面。 ANT_URL 是那些需要认证的 API。 ROLE 是指有什么用户权限才能使用该 API。

当这个文件发到我们及完成处理后, 请开发者参考以下文件做认证:

```

<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/1.12.0/jquery.min.js"></script>
<script>
var server = "https://localhost:12345";
var username = 'user_account';
var password = 'user_password';
var token = "";
$.ajax
({
  type: "POST",
  url: server + "/oauth/token",

```

```
contentType: "application/x-www-form-urlencoded",
data: "grant_type=password&username="+username+"&password="+password+"&client_id=user_account",
success: function (data){
    console.debug(data);
    token = data['access_token'];
    $('#token').html('token:' + token);
    getFaqs();
}
});

function getFaqs(){
    if ( token == undefined )
    {
        alert('auth token is not ready!!!');
        return;
    }
    $.ajax
    ({
        type: "GET",
        url: server + "/api/faq/all?access_token=" + token,
        success: function (data){
            console.debug(data);
            $('#result').text(JSON.stringify(data).replace(new RegExp('{','g'),'','\n'));
        }
    });
}
</script>
</head>
<body>
Testing page
<div id="token"></div>
<div>
<div><pre id="result"></pre></div>
</div>
</body>
</html>
```

在以上 HTML 可以看到, 开发者需用 API/oauth/token 做认证拿取 token, 之后只要在每次使用接口时, 在 URL 上加上 access_token 参数即可。

9.2 分类跳转 (Category Switching)

针对上下文对话，引擎提供分类跳转功能。当引擎识别到用户问题的意图时，引擎可以将当前对话跳转到指定的对话流程，用户的下一次提问就可以进入到指定的对话流程。结合训练数据，引擎可以做到对话流程的灵活跳转。

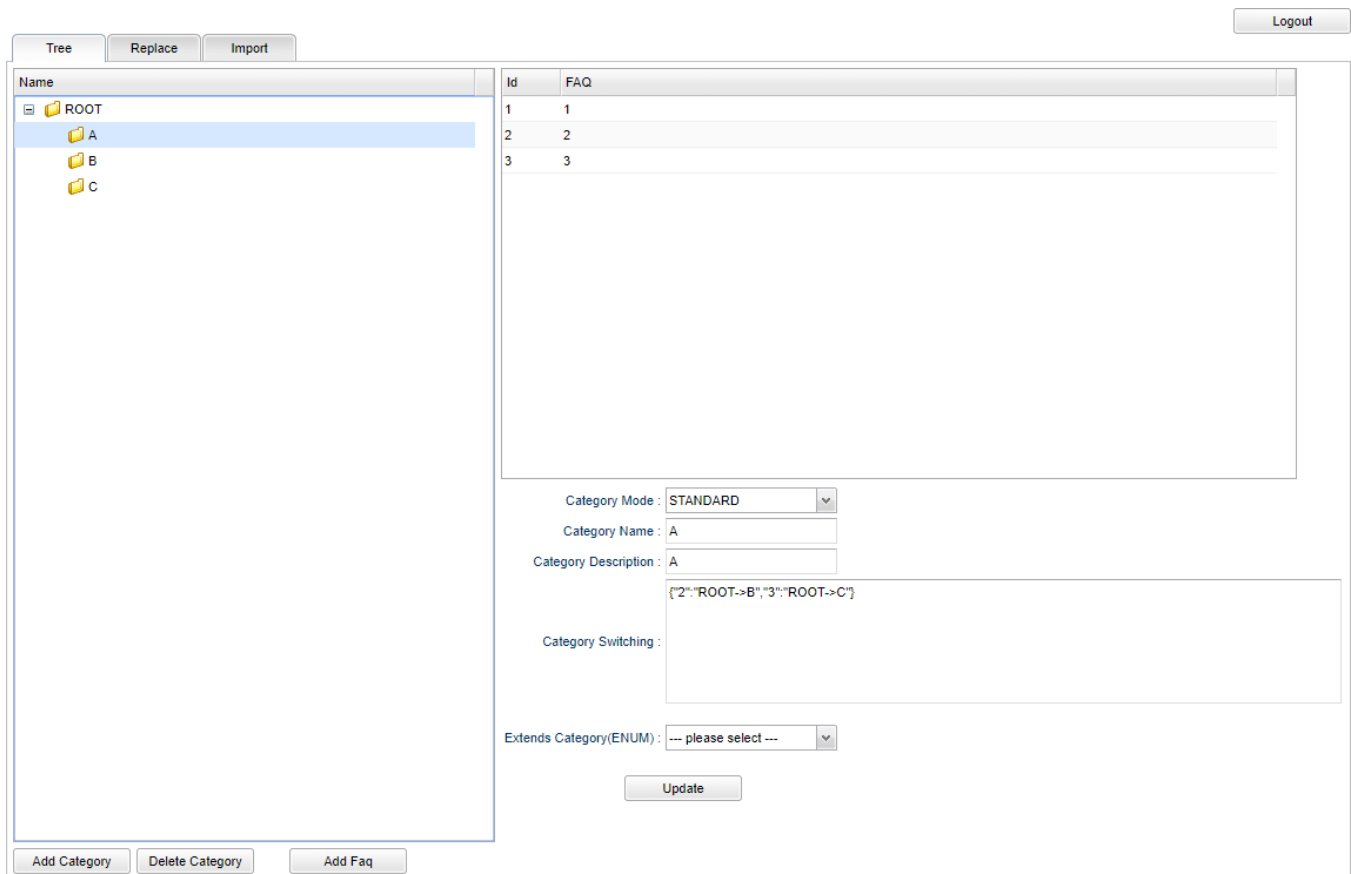
分类跳转功能使用的场景是当用户在某一分类下提问与此分类不相干的问题时，开发者在该分类下添加一条知识点用于匹配该类问题，然后将该知识点指向需要的目标分类。具体配置方法如下：

例如 ROOT 分类下有 A、B、C 三个标准型分类。分类 A 下有 3 个 FAQ，分类 B 下有 2 个 FAQ，分类 C 下有 2 个 FAQ，Faqid 依次为 1 至 7。假设引擎将用户的问题识别为 FAQ 2 时，引擎跳转到分类 B 中，下次用户的问题跳转分类 B 中进行匹配；同理引擎识别到 FAQ 3 时，将跳转到分类 C。

分类跳转功能需要在 Category Switching 栏里用 JSON 格式指明 Faqid 和目标分类的 QID，格式如下：

```
{"<Faqid_1>":"<QID_1>","<Faqid_2>":"<QID_2>"}
```

其中，Faqid 为当引擎匹配到哪一条 FAQ，分类才需要跳转；QID 为需要跳转的目标分类的分类路径，各分类之间用“->”间隔。



The screenshot shows a web interface for managing categories and FAQs. On the left, a tree view shows a hierarchy: ROOT (expanded) -> A (selected), B, and C. On the right, a table lists FAQs with columns 'Id' and 'FAQ'. Below the table, there are input fields for 'Category Mode' (set to STANDARD), 'Category Name' (A), and 'Category Description' (A). The 'Category Switching' field contains a JSON string: {"2":"ROOT->B","3":"ROOT->C"}. At the bottom, there is an 'Extends Category(ENUM)' dropdown set to '--- please select ---' and an 'Update' button.

Id	FAQ
1	1
2	2
3	3

Category Mode: STANDARD
Category Name: A
Category Description: A
Category Switching: {"2":"ROOT->B","3":"ROOT->C"}
Extends Category(ENUM): --- please select ---
Update

如上图所示，在本例中，选中需要进行跳转的分类“ROOT->A”，在“Category Switching”栏按如下配置：

```
{"2":"ROOT->B","3":"ROOT->C"}
```

分类跳转的效果需要结合训练数据和 API 接口一起实现。在建立好分类和知识点，以及配置好分类跳转功能后，用户要针对需要跳转的知识点，录入对应的训练数据对引擎进行训练。建议使用 API 接口 9.2“指定分类的单次文字意图识别”，指定分类路径录入需要的训练数据。本例中，训练数据如下：

Update Date From: To:
 Question: Faqid: Expected Faqid: Responseld:

Create Date From: To:
 Confidence Min: Max:

☐ Include training data ☐ Mismatch FAQID

« 1 » Go

ID	Timestamp	QID	Question	Faqid	Expected Faqid	CL%	Responseld
10448	15-11-2017 12:29:29	ROOT->A	1	-1	1	0.00	1111
10449	15-11-2017 12:29:32	ROOT->A	2	-1	2	0.00	1112
10450	15-11-2017 12:29:38	ROOT->A	3	-1	3	0.00	1113
10451	15-11-2017 12:29:46	ROOT->B	4	-1	4	0.00	1114
10452	15-11-2017 12:29:53	ROOT->B	5	-1	5	0.00	1115
10453	15-11-2017 12:30:20	ROOT->C	6	-1	6	0.00	1116
10454	15-11-2017 12:30:23	ROOT->C	7	-1	7	0.00	1117

引擎训练完成后，开发者可以用接口 9.2“指定分类的单次文字意图识别”，在指定的分类里进行查询，当引擎匹配到设置了分类跳转的知识点时，返回的 JSON 结果里会包含“followUp”参数，参数值即为需要跳转的分类路径。下一次的查询时，开发者指定在“followUp”中的分类中进行查询，对话流程就会跳转到“followUp”中的分类，这样就可以做到对话流程跳转的效果。

在本例中，使用 9.2“指定分类的单次文字意图识别”进行查询的示例如下：

1. 指定分类进行查询，引擎匹配到未配置分类跳转的知识点，返回结果中不会包含“followUp”参数。

URL: <http://192.168.1.103:25000/queryj/x/ROOT->A/2111/1>

结果：

```
{
  "responseld": "2111",
  "answer": "1",
  "confidence": "99.55",
  "question": "1",
  "y": "1"
}
```

2. 指定分类进行查询，引擎匹配到配置了分类跳转的知识点，返回结果中会包含“followUp”参数：

URL: <http://192.168.1.103:25000/queryj/x/ROOT->A/2112/2>

结果：

```
{
  "responseld": "2112",
  "answer": "2",
  "confidence": "99.55",
  "question": "2",
  "followUp": "ROOT->B",
  "y": "2"
}
```

当引擎匹配到知识点 2 时，下一次查询只要在“followUp”参数指定的分类路径“ROOT->B”下进行，引擎就会将用户的问题在“ROOT->B”下进行匹配，从而达到分类跳转的效果。

分类跳转功能对所有分类均适用，本例仅以标准型为例，其他分类同理。对于顺序型、树状型、细化型三种上下文分类，分类本身的知识点也可以进行分类跳转配置，使用方法与普通分类下的知识点相同。对于枚举型中分类跳转的知识点，需要在引用该枚举型分类的顺序型分类中进行配置。

开发者可以使用以下例子为知识库导入引擎：

```
ID,ACTION,CATEGORY,QUESTION,ANSWER,ATTACHMENT
1,ADD,ROOT->A,1,1
2,ADD,ROOT->A,2,2
3,ADD,ROOT->A,3,3
4,ADD,ROOT->B,4,4
5,ADD,ROOT->C,5,5
#START OF CATEGORY
#MODE,ACTION,PATH,DESC,FAQID
STANDARD,CAT,ROOT,{"SWITCHING":null,"DESC":null,"VER":"1.0"},
STANDARD,CAT,ROOT->A,{"SWITCHING":{"2":"ROOT->B","3":"ROOT->C"},"DESC":null,"VER":"1.0"},
STANDARD,CAT,ROOT->B,{"SWITCHING":null,"DESC":null,"VER":"1.0"},
SRANDARD,CAT,ROOT->C,{"SWITCHING":null,"DESC":null,"VER":"1.0"},
```

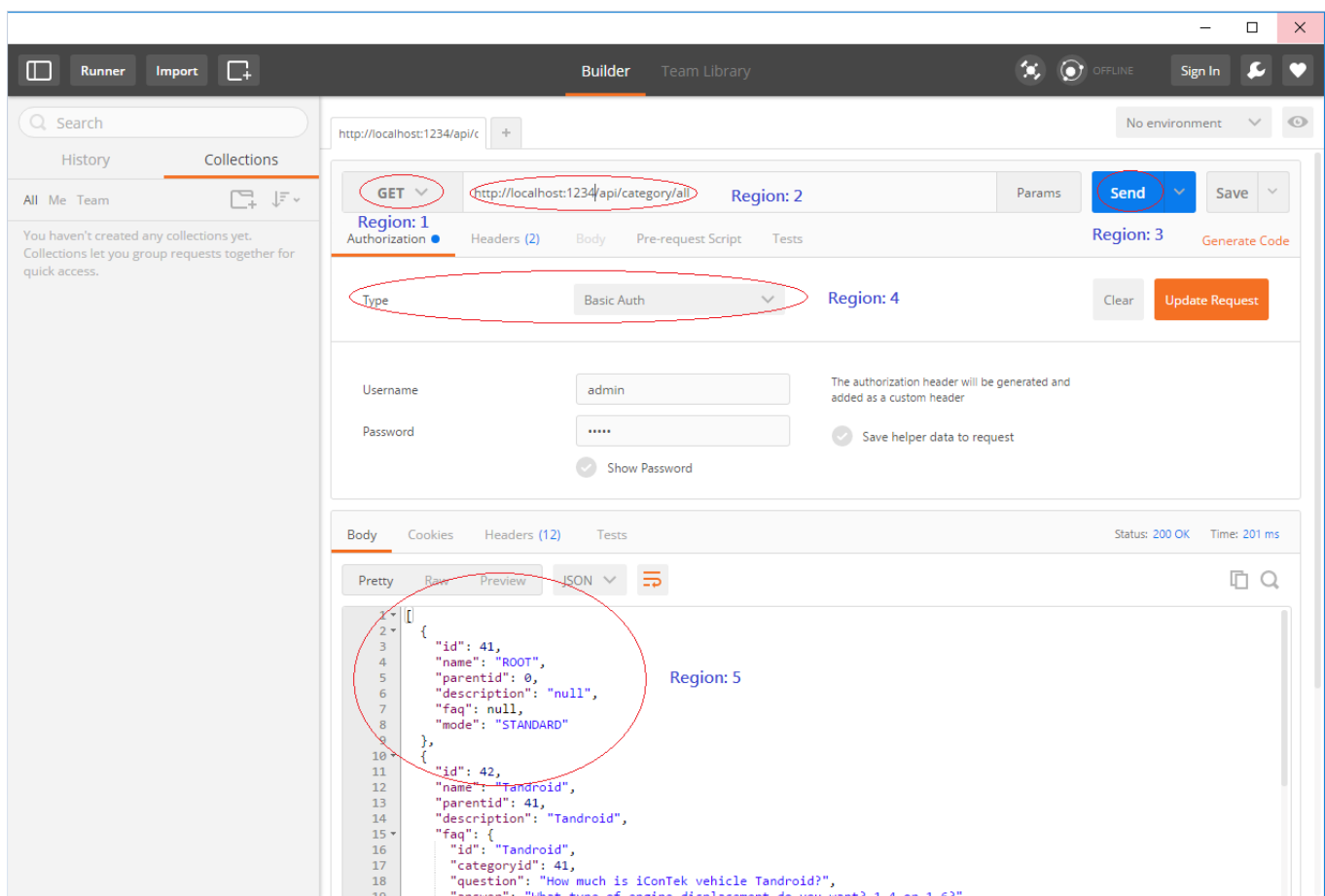
开发者可以使用以下例子为训练数据导入引擎：

1	1	ROOT->A
2	2	ROOT->A
3	3	ROOT->A
4	4	ROOT->B
5	5	ROOT->C

附录一： 测试引擎教学

开发商收到 NLP Engine API Manual, 可能会想立即测试引擎的功能, 但又不想立即开发应用软件; 又或软件开发时, 有另一组人要同时训练引擎。这个时候, 我们建议开发商下载 Restful Client 测试工具。当然市面上有很多 Restful Client 的测试工具, 开发商可自行选择顺手的工具, 而我们这个教学, 主要是针对 POSTMAN 为引擎测试工具。

NLP Engine API 上的功能, 大致只有 HTTP GET 同 HTTP POST 两种请求指令, 开发商安装好 POSTMAN 后, 如下图所示, 用户可选“Region: 1”的下拉菜单, 在 Postman 选择用 GET, 还是用 POST 请求传送到引擎, 如图一: <简单 POSTMAN HTTP GET(无认证)范例>所示, 这就是一个 HTTP GET 的演试例子。



图一: 简单 POSTMAN HTTP GET(无认证)范例

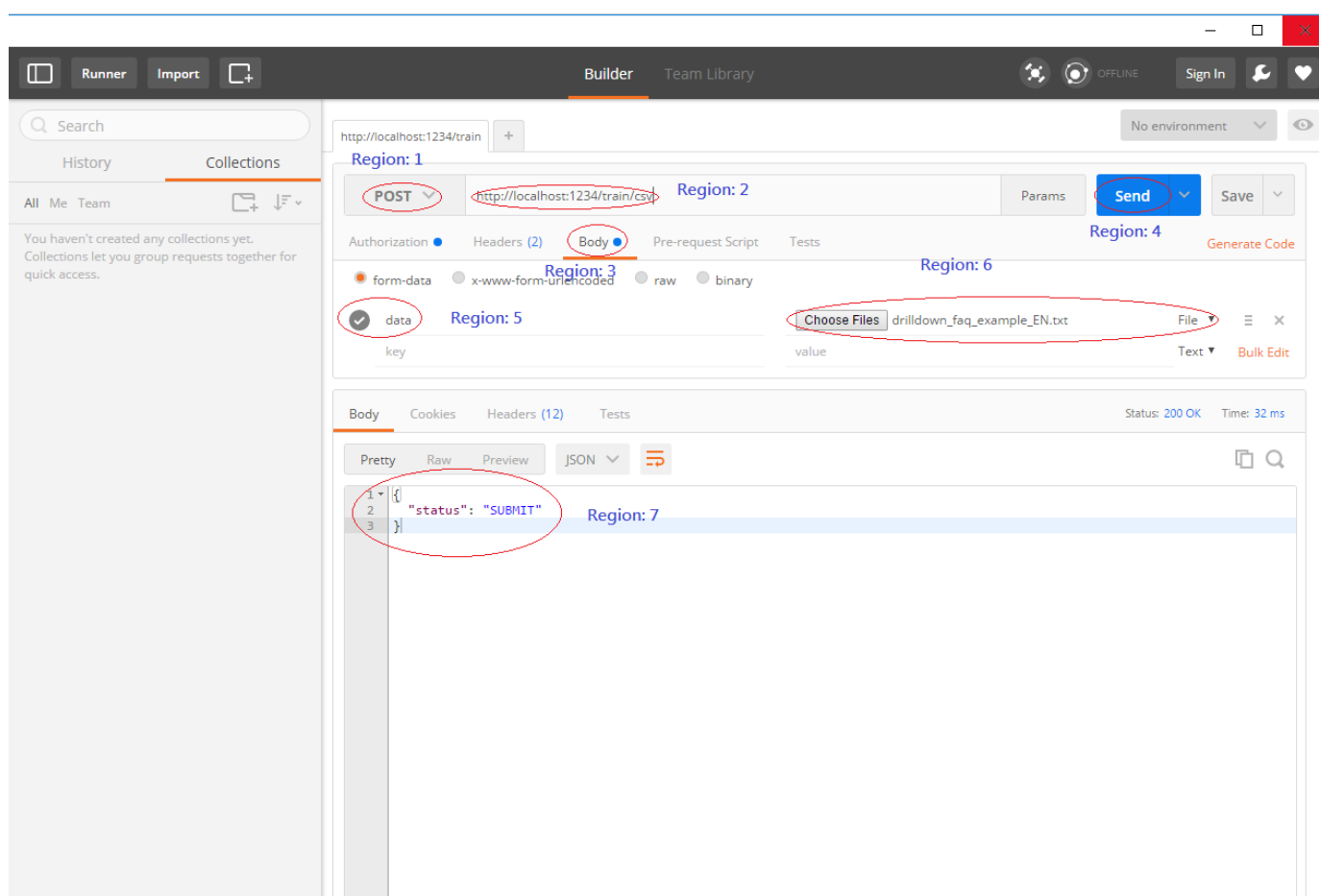
在“Region: 2”填上适当的 URL, 如上图所示, 这是用到本文件单元 3.1 <提取所有知识库分类纪录>功能, 开发人员只需将引擎 API 的 URL, 复制到这个位置即可。如何合并引擎的 Server Address, Port Address 及 API URL 制造这个访问引擎的 URL, 请开发人员小心阅读本文件的单元一。如用 HTTPS 端口, 请小心在 URL 加上“https”字样。

以上例子是简单认证 HTTP GET 范例, 所以在“Authorization 的分页中, 在“Region: 4”选择“Basic Auth”即可。

以上填妥后, 就可按“Region: 3”的“Send”键, 然后在“Region: 5”, 就会出现系统响应的相关答案。

基本上本文件的 HTTP GET 功能, 可以这样就引用得到。

当然 NLP Engine API 会有很多 HTTP POST 的请求指令, 而且大部份也会和 CSV 档案上传有关, 下图所示就是以本文件的单元 5.1 <批量导入训练数据>为蓝本, 示范如何在 POSTMAN 中编辑 HTTP POST 上传 CSV 档案的请求指令:



图二: POSTMAN HTTP POST 范例

从以上例子看到, 在“Region: 1”的下拉菜单, 选择了“POST”, 然后在“Region: 2”上补上了“/train/csv”的 URL。之后点击“Region: 3”的“Body”分页, 在“Body”分页中, 在“Region: 5”填上“data”, 之后在“Region: 6”的下拉菜单, 选择“File”, 再按“Choose file”键, 选择相应的 CSV 上传档案。

以上填妥后, 就可按“Region: 4”的“Send”键, 如果没有什么问题, 那在“Region: 7”就会出现引擎成功上传的响应。

基本上 POSTMAN 中的 HTTP GET 和 HTTP POST 的功能已演练一次, 只要学懂这两种动作, 已经能用到大部份引擎功能, 而无须开发商写任何程序。其他相关的 POSTMAN 功能, 用户可自行在网上寻找 POSTMAN 的教学研究。

附录二： 错误信息说明

当调用 API 接口时出现错误, 引擎会返回错误信息, 以下为错误信息的说明:

错误信息	说明
ADD_FAQ_CANNOT_BE_INVALID	与分类关联的知识点无效。
ADD_FAQ_CANNOT_BE_NULL	与分类关联的知识点不能为空。
CATEGORY_NOT_FOUND	未找到该分类。
DUPLICATE_CATEGORY_NAME	该分类名称重复。
DUPLICATE_FAQ	该知识点重复。
EMPTY_ANSWER	该知识点的答案为空。
EMPTY_ID	该知识点的 ID 为空。
EMPTY_INPUT	该输入值为空。
EMPTY_NAME	该分类的名称为空。
EMPTY_QUESTION	该知识点的答案为空。
FAQ_NOT_FOUND	未找到该知识点。
INVALID_CATEGORY	该分类无效。
INVALID_ID	该知识点 ID 无效。
PARENT_NOT_FOUND	未找到父分类。
UPDATE_FAQ_CANNOT_BE_INVALID	与分类关联的知识点无效。
UPDATE_FAQ_CANNOT_BE_NULL	与分类关联的知识点不能为空。

注意：引擎的错误信息包括但不限于以上列表。如遇到列表之外的错误信息, 请与我们技术人员联系。

附录三： 语音活性检测（VAD）参数说明

开发者在使用接口 **6.3 启动数据流语音识别** 时，可以根据实际需求使用语音活性检测（VAD）参数进行开发。具体参数说明如下：

VAD 基本参数：

参数名	取值范围	默认值	建议值	说明
sampleRate	8000, 16000, 32000 (Hz)	8000	实际采样率	语音采样率，语音格式为 16-bit Signed PCM。电话语音通常为 8000 (Hz)，PC 语音通常为 16000 或 32000 (Hz)。
MinActive-Speech-Duration	100-2000 (ms)	200	200	录音中有效语音需要连续的最短时间。若用户可能回答简短答案，此值不宜设置过大，否则 VAD 将无法快速检测到用户语音。
Max-Speech-Duration	0-20000 (ms)	15000 (ms)	15000 (ms)	最长录音时间，若录音超过此时间，VAD 将会中止并返回。此值对应 MRCP 中的 Recognition Timeout 参数。
End-Silence-Duration	500-3000 (ms)	800	1100	尾部静默时间，当一段录音中的尾部静默达到此值，并且有效语音达到 MinActiveSpeechDuration 时，VAD 判断用户说话结束。
Speech-Timeout	100-500 (ms)	150 (ms)	150 (ms)	从静音到有效语音的状态转换时间，此值不建议修改。
Silence-Timeout	100-500 (ms)	300	300	从有效语音到静音的状态转换时间，此值不建议修改。
No-Input-Timeout	0-30000 (ms)	5000	5000	录音开始之后，保持静默的最长时间。超过此时间之后，VAD 将会中止并返回用户未说话。

VAD 灵敏度通过 MinActive-Speech-Duration 和 End-Silence-Duration 组合设置，MinActive-Speech-Duration 和 End-Silence-Duration 值越小，灵敏度越高，但是通常情况下 MinActive-Speech-Duration 值不宜高于 500ms，End-Silence-Duration 不宜低于 100ms，否则将会产生大量误检率，提早中断用户说话。

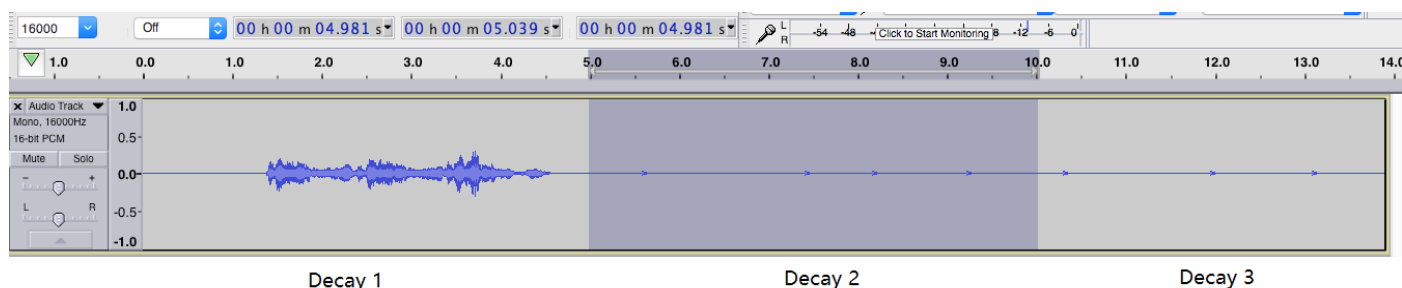
多段 VAD 参数:

参数名	取值范围	默认值	建议值	说明
Decay-Enable	0,1	1	1	是否启用多段 VAD 检测, 1 为启用, 0 为禁用。建议启用。
Decay-Time1	0-30000	4500	5000	第 1 段 VAD 检测结束时间点
Decay-Time2	0-30000	6000	10000	第 2 段 VAD 检测结束时间点
Decay-Time3	0-30000	15000	15000	第 3 段 VAD 检测结束时间点
Decay-Silence-Duration1	500-3000 (ms)	1500 (ms)	1500 (ms)	第 1 段 VAD 检测中的尾部静音长度
Decay-Silence-Duration2	500-3000 (ms)	1100 (ms)	1100 (ms)	第 2 段 VAD 检测中的尾部静音长度
Decay-Silence-Duration3	500-3000 (ms)	700 (ms)	700 (ms)	第 3 段 VAD 检测中的尾部静音长度
Decay-Active-Duration1	100-2000 (ms)	600 (ms)	600 (ms)	第 1 段 VAD 检测中的最短有效语音长度
Decay-Active-Duration2	100-2000 (ms)	300 (ms)	300 (ms)	第 2 段 VAD 检测中的最短有效语音长度
Decay-Active-Duration3	100-2000 (ms)	200 (ms)	200 (ms)	第 3 段 VAD 检测中的最短有效语音长度

多段 VAD 灵敏度通过 Decay-Active-Duration 和 Decay-Silence-Duration 两类参数组合设置, Decay-Active-Duration 和 Decay-Silence-Duration 值越小, 灵敏度越高, 但是需要设置 Decay-Active-Duration 不低于 100ms, Decay-Silence-Duration 不低于 100ms, 否则将会产生大量误检率, 提早中断用户说话。

实际使用中, 如果在用户说话时开始录音, 可以将第一段 VAD 检测长度设置为提示音长度, 此时可以设置较低的灵敏度 (如: Decay-Silence-Duration1 = 1500ms, Decay-Active-Duration1 = 1000ms); 第二段 VAD 检测长度设置为提示音长度 + 5 秒, 设置正常灵敏度 (如: Decay-Silence-Duration2 = 1100ms, Decay-Active-Duration2 = 300ms); 第三段 VAD 检测长度设置为提示音长度 + 5 秒, 设置为较高灵敏度 (如: Decay-Silence-Duration3 = 700ms, Decay-Active-Duration3 = 200ms)。

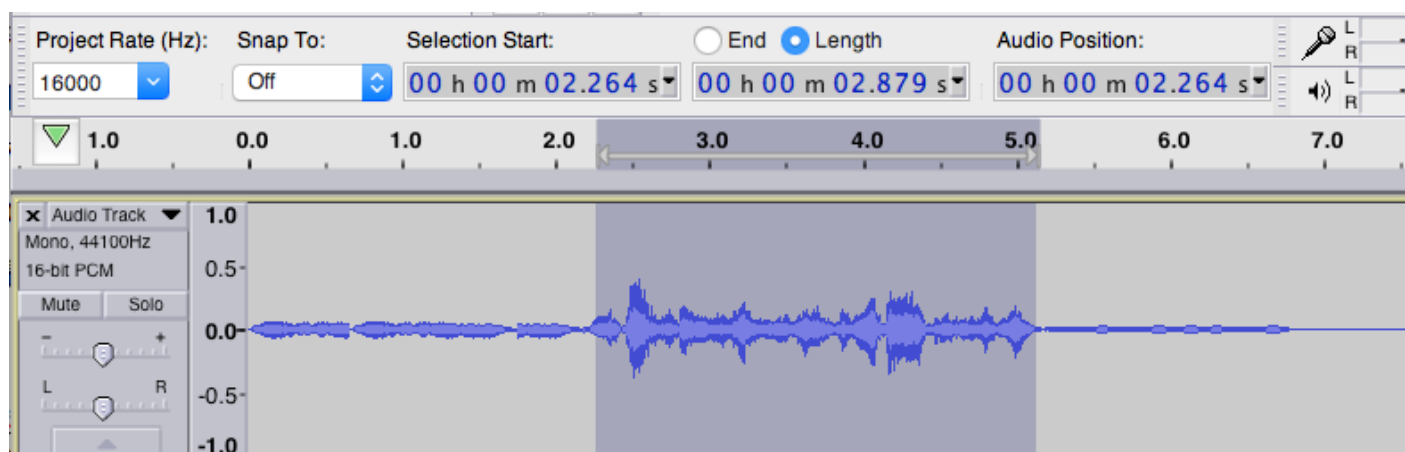
我们可以将需要进行 VAD 检测的语音进行分段检测, 设置不同的灵敏度来检测用户是否结束说话。多段 VAD 检测例子如下, 每段为 5 秒, 共 15 秒:



自适应背景噪声参数：

参数名	取值范围	默认值	建议值	说明
Adaptive-Level-Enable	0, 1	1	1	是否启用自适应背景噪音阈值检测，建议设置为 1（启用），可以改善在一定背景噪音环境下的静音检测。
Adaptive-Level-Duration	100-800 (ms)	300 (ms)	200 (ms)	自适应背景噪音检测的时长，当 a2x.adaptiveLevelEnable=1 时，该值有效，背景噪音检测时间为录音开始至该长度。
Adaptive-Level-Coef	1-5	2	2	自适应噪声调整系数，取值范围 1-5，默认值 2。这个值越大，背景噪音调整幅度越大。

下图是一个利用自适应背景噪声来改善 VAD 检测的例子，在语音开始阶段，背景存在一定噪音，VAD 会自动将静音判定阈值自动调高，避免背景噪音被误判为有效语音，当语音在约 2.2 秒出现时，VAD 可以检测到有效语音，在大约 5.1 秒时，VAD 检测到有效语音结束。



附录四： 语音转文字功能说明

除了语音意图识别功能之外，引擎还提供语音转文字功能，该功能可以将输入的语音转换成预设语种的文字，目前仅支持中文和英文。请注意，语音转文字功能非标准功能，需在指定版本引擎中开启才可使用。并且该功能与语音意图识别功能不可以同时使用，当语音转文字功能开启时，语音意图识别功能自动失效。

以下为语音转文字功能的介绍和使用方法：

1. 上传语音文件进行语音转文字

URL	/api/uaudio/file/text
HTTP Method	Post
功能	上传特定格式的语音文件到引擎，引擎会将该语音文件的内容转换成文字。
输入	File (form-data name="data") 文件类型：WAV 文件，语音格式为 signed 16 bit pcm(raw) mono 格式
输出	数据类型：纯文本 以纯文本形式输出转换后的文字。

引擎提供通过传送语音流的方式进行语音转文字，分为单次转换和连续转换，两种转换方式都是基于 Websocket 协议传送语音流。Websocket 请求分为指令传送部分和语音流传送部分，其中指令传送部分传送文字指令，用于控制 Websocket 连接和处理返回结果；语音流部分则传送语音流，供引擎进行语音转换。

2. 语音流单次转换

URL	/api/uaudio/websocket
Request Method	Websocket (支持 ws 和 wss)
功能	向引擎传送语音流，进行单次语音转文字。当引擎接收到停止指令或者检测到静默时，返回本次语音转换的文字结果。
文字部分 – 开启语音流单次转换	
输入	数据类型：JSON 对象 - ACTION: START，发送开启传送语音流指令。使用 START 开启 websocket 连接之后，开发者需要向引擎传送语音流，当语音流传送结束，开发者需要另外发送 STOP 或者 CANCEL 关闭 websocket 连接。 - ASR: 开启或关闭语音转文字功能，true 或 false。 - responseId: 该段语音流的唯一标识号。需要开发者指定，且大小写敏感。 - sampleRate (选填)：语音流的采样率，默认值为 8000，只支持 8000 至 44100。 除此之外，引擎还提供语音活性检测 (VAD)参数供开发使用，该部分参数为选填。具体参数的使用方法参见「附件三：语音活性检测 (VAD) 参数说明」。 【范例】 <pre>{"ACTION": "START", "ASR": "true", "sampleRate": 44100, "responseId": "<UNIQUE_ID>"}</pre>
输出	数据类型：JSON 对象 - STATUS: 在传送语音流的时候，STATUS 必须为“RECORDING”，如 STATUS 为其他状态，则代表传送语音予引擎的对话结束。如 STATUS 为“FAIL”状态，则代表不能开始识别。 - ACTION: 在传送语音流给引擎时，ACTION 必须为“START”，“STOP”则代表语音传送结束，引擎会返回识别结果。“CANCEL”则代表取消本次语音识别，引擎不会返回识别结果。 - HANDLE: 整数，如果 HANDLE 小于 0 代表引擎现在繁忙，不能接收语音流。 【范例】 <pre>{"STATUS": "RECORDING", "ACTION": "START", "HANDLE": <HANDLE_ID>}</pre>
文字部分 – 停止语音流单次转换	

输入	数据类型：JSON 对象 - ACTION: STOP, 发送停止传送语音流指令。开发者需要传送“STOP”指令来停止传送中的语音流, 收到停止指令后, 引擎会输出转换后的文字结果。 【范例】 <pre>{"ACTION": "STOP"}</pre>
输出	数据类型：JSON 对象 - STATUS: 返回引擎当前状态, “true”或“false”。 - ACTION: 值为“STOP”, 表示语音转换结束。 - RESULT: 引擎语音转文字的结果, 以 JSON 格式输出。 - txt: 该段语音流转换后产生的文字。 - responseld: 该段语音流的唯一标识号。需要开发者指定, 且大小写敏感。 【范例】 <pre>{"STATUS": true, "ACTION": "STOP", "RESULT": {"txt": "<OUTPUT TEXT>", "responseld": "<responseld from INPUT>"}}</pre>
文字部分 – 取消语音流单次转换	
输入	数据类型：JSON 对象 - ACTION: CANCEL, 发送取消传送语音流指令。开发者需要传送“CANCEL”指令来取消传送中的语音流, 引擎不会返回任何信息。 【范例】 <pre>{"ACTION": "CANCEL"}</pre>
输出	没有任何输出
语音流部分 – 传送语音流	
输入	数据格式：Signed 16bit PCM (raw) 单声道 (Mono) 格式的语音流, 采样率为 8000 到 44100 之间。 如有必要, 开发者需要自行参考 websocket 相关协议 (RFC6455) 。
输出	数据格式：JSON 对象 主要有以下三种形式： - 返回语音转换结果：引擎接收到停止指令或检测到静音。返回结果与文字部分 – 停止语音流单次转换的返回结果完全一致。 - 返回失败信息：引擎返回失败信息。该情况通常由于在发送开启指令之前传送语音流。 - 无任何返回信息：该结果表示引擎仍在接收语音流。 【范例 1 – 返回语音转换结果】

```
{"STATUS":true,"ACTION":"STOP","RESULT":{"txt":"<OUTPUT TEXT>","responseId":"<responseId from INPUT>"}}
```

【范例 2 – 返回失败信息】

```
{"STATUS":"FAIL","ACTION":"WRITE","MESSAGE": "NOT_START"}
```

用 websocket 连接系统进行语音流单次转换时, 在代码内基本上是以下流程:

1. init websocket connection
2. calling start api
3. keep writing to the stream until receive message until end
4. close connection

Message call back:

```
if ( "START" == action ){  
    if ( handle <= 0 ){  
        alert("Cannot start recording !!!" + handle);  
    }  
}else if ( "STOP" == action ){  
    stop_stream();  
    handleResultList(["RESULT"]);  
}else if ( "WRITE" == action ){  
    if ( "FAIL" == status ){  
        stop_stream();  
        alert("cannot handle process");  
    }  
}  
}
```


3. 语音流连续转换

URL	/api/dictation/websocket
Request Method	Websocket (支持 ws 和 wss)
功能	向引擎传送语音流, 进行连续语音转文字。在传送语音流期间, 每一次引擎检测到静默或者语音流最大长度达到 20 秒时, 引擎会返回转换后的文字结果。在此期间, 引擎会持续接收传入的语音流进行语音转换, 并返回转换后的文字结果, 直到引擎接收到停止指令或再没有语音流传入。
文字部分 – 开启语音流连续转换	
输入	数据类型: JSON 对象 - ACTION: START, 发送开启传送语音流指令。使用 START 开启 websocket 连接之后, 开发者需要向传送语音流, 当语音流传送结束, 开发者需要另外发送 STOP 或者 CANCEL 关闭 websocket 连接。 - responseld: 该段语音流的唯一标识号。需要开发者指定, 且大小写敏感。 - sampleRate (选填): 语音流的采样率, 默认值为 8000, 只支持 8000 至 44100。 除此之外, 引擎还提供语音活性检测 (VAD)参数供开发使用, 该部分参数为选填。具体参数的使用方法参见「附件三: 语音活性检测 (VAD) 参数说明」。 【范例】 <pre>{"ACTION": "START", "ASR": "true", "sampleRate": 44100, "responseld": "<UNIQUE_ID>"}</pre>
输出	数据类型: JSON 对象 传送语音流之前和传送语音流之后的输出结果不同。传送语音流之前, 返回开启指令的结果, 传送语音流之后, 引擎会根据传入的语音流输出相应结果。 传送语音流之前: - STATUS: 在传送语音流的时候, STATUS 必须为“RECORDING”, 如 STATUS 为其他状态, 则代表传送语音予引擎的对话结束。如 STATUS 为“FAIL”状态, 则代表不能开始识别。 - ACTION: 在传送语音流给引擎时, ACTION 必须为“START”, “STOP”则代表语音传送结束, 引擎会返回识别结果。“CANCEL”则代表取消本次语音识别, 引擎不会返回识别结果。 - HANDLE: 整数值, 如果 HANDLE 小于 0 代表引擎现在繁忙, 不能接收语音流。 【范例】 <pre>{"STATUS": "RECORDING", "ACTION": "START", "HANDLE": <HANDLE_ID>}</pre> 传送语音流之后: - ACTION: 该参数值为 MESSAGE。

	- message: 引擎语音转文字的结果，以 JSON 格式输出。 - responseId: 该段语音流的唯一标识号。需要开发者指定，且大小写敏感。 - sentId: 连续转换产生的文字结果的序号。多个文字结果返回是可能不按照顺序排列。 - utterName: 每段返回的文字结果的唯一标识号。 - result: 该段语音流转换后产生的文字。 - createDate: 该段语音流发送时的时间戳。 - status: 返回引擎当前状态，“true”或“false”。 【范例】 <pre>{ "ACTION": "MESSAGE", "message": [{ "responseId": "<UNIQUE_ID>", "sentId": "<sentId>", "utterName": "<utterName>", "result": "你好 你好", "createDate": "1559083927876" }], "status": "true" }</pre>
文字部分 – 停止语音流连续转换	
输入	数据类型：JSON 对象 - ACTION: STOP，发送停止传送语音流指令。开发者需要传送“STOP”指令来停止传送中的语音流，引擎会输出转换后的文字结果。 【范例】 <pre>{ "ACTION": "STOP" }</pre>
输出	数据类型：JSON 对象 - feedback: 整数值。该值大于 0 时，表示成功停止语音流连续转换。 - STATUS: 返回引擎当前状态，“true”或“false”。 - ACTION: 值为“STOP”，表示语音识别结束。 【范例】 <pre>{ "feedback": "<feedback>", "STATUS": "STOP", "ACTION": "STOP" }</pre>
语音流部分 – 传送语音流	
输入	数据格式：Signed 16bit PCM (raw) 单声道 (Mono) 格式的语音流，采样率为 8000 到 44100 之间。 如有必要，开发者需要自行参考 websocket 相关协议 (RFC6455) 。
输出	数据格式：JSON 对象 主要有以下两种形式： - 返回失败信息：引擎返回失败信息。该情况通常由于在发送开启指令之前传送语音流。 - 无任何返回信息：该结果表示引擎仍在接收语音流。 【范例 – 返回失败信息】 <pre>{ "STATUS": "FAIL", "ACTION": "WRITE", "MESSAGE": "NOT_START" }</pre>

用 websocket 连接系统进行语音流连续转换时, 在代码内基本上有以下流程:

1. init websocket connection
2. calling start api
3. keep writing to the stream
4. keep receiving text from time to time until close command issued or nothing stream into the port
5. close connection

Message call back:

```
if ( "START" == action ){  
    if ( handle <= 0 ){  
        alert("Cannot start recording !!!" + handle);  
    }  
}else if ( "STOP" == action ){  
    stop_stream();  
    handleResultList(["RESULT"]);  
}else if ( "WRITE" == action ){  
    if ( "FAIL" == status ){  
        stop_stream();  
        alert("cannot handle process");  
    }  
}  
}
```

如想索取简单利用 websocket 作语音意图识别的 JAVA 代码例子,可连络我们索取相关资料。